

Москва 2026

ПЛАН РАБОТЫ НАД КУРСОВЫМ ПРОЕКТОМ

[illegible]

Преподаватель: / _____ / Лаврененко Илья Станиславович /
подпись ФИО

Студент: /  / Залужный Никита Дмитриевич, 231-324 /
подпись ФИО, группа

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)
КАФЕДРА СМАРТ-ТЕХНОЛОГИИ

Курсовой проект
по дисциплине «Системы инженерного анализа»
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль)
«Интеграция и программирование в САПР»

Преподаватель: / _____ / Лаврененко Илья Станиславович /
подпись *ФИО*

Студент: / _____ / Залужный Никита Дмитриевич, 231-324 /
подпись *ФИО, группа*

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное автономное образовательное учреждение
высшего образования

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ

по направлению 09.03.01 Информатика и вычислительная техника

Образовательная программа (профиль) «Интеграция и программирование в
САПР»

ТЕМА КУРСОВОГО ПРОЕКТА	Разработка веб-приложения для расчета течения воздуха через канал заданной формы средствами OpenFOAM и ParaView.
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Расчёт и визуализация стационарного течения воздуха через канал заданной формы.
Основные функции	1. Редактирование параметров модели. 2. Запуск расчёта. 3. Вывод результата.
Используемые технологии и платформы	OpenFOAM 9, ParaView, Node.js, VS Code.
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Создание геометрии по заданному варианту. 2. Параметризация геометрии. 3. Автоматизация формирования и запуска расчетной модели OpenFOAM. 4. Создание веб-сервера и веб-клиента.
Состав технической документации	Листинги программного кода
Состав графической части	Экранные формы приложения и изображения результатов расчёта.

ПЛАН РАБОТЫ НАД КУРСОВЫМ ПРОЕКТОМ

ЗАДАЧИ	НЕДЕЛИ											
	1	2	3	4	5	6	7	8	9	10	11	12
Обзор предметной области и анализ аналогов												
Изучение OpenFOAM 9, ParaView и нормативной документации												
Проектирование архитектуры приложения												
Разработка геометрии и генерация сетки												
Настройка расчётной модели и решателя												
Разработка веб-клиента и веб-сервера												
Тестирование и проведение расчётов												
Оформление пояснительной записки												

Преподаватель: / _____ / Лаврененко Илья Станиславович /
подпись *ФИО*

Студент: / _____ / Залужный Никита Дмитриевич, 231-324 /
подпись *ФИО, группа*

АННОТАЦИЯ

Курсовой проект состоит из пояснительной записки и программного приложения. Данная работа посвящена разработке веб-приложения по расчёту течения воздуха через канал заданной формы средствами OpenFOAM и ParaView. Расчётной средой является воздух. В состав пояснительной записки входят введение, обзор научно-технической литературы, аналитический расчёт и моделирование в OpenFOAM, анализ и подготовка геометрии, описание программного кода создания модели, разработка приложения, тестирование приложения и сравнение результатов, а также заключение.

Работа состоит из шести разделов. В пояснительную записку включены рисунки и листинги программного кода. В приложениях приведён полный листинг основных файлов разработанного приложения.

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	7
ВВЕДЕНИЕ	8
1 ОБЗОР И АНАЛИЗ НАУЧНО-ТЕХНИЧЕСКОЙ ЛИТЕРАТУРЫ	10
2 АНАЛИТИЧЕСКИЙ РАСЧЕТ И МОДЕЛИРОВАНИЕ В OPENFOAM	12
2.1 Исходные данные и аналитический расчёт.....	12
2.2 Построение расчётной сетки	15
2.3 Настройка расчётной модели в OpenFOAM	15
2.4 Результаты численного моделирования	16
3 АНАЛИЗ И ПОДГОТОВКА ГЕОМЕТРИИ.....	19
3.1 Исходные данные	19
3.2 Подготовка геометрии	19
4 НАПИСАНИЕ КОДА СОЗДАНИЯ МОДЕЛИ.....	21
4.1 Создание топологии	21
4.2 Генерация файла расчётной сетки	21
4.3 Генерация начальных и граничных условий.....	22
4.4 Настройка решателя	23
5 РАЗРАБОТКА ПРИЛОЖЕНИЯ	25
5.1 Архитектура приложения.....	25
5.2 Разработка клиентской части.....	25
5.3 Разработка серверной части.....	27
6 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ И СРАВНЕНИЕ РЕЗУЛЬТАТОВ	29
6.1 Тестирование приложения	29
6.2 Сравнение аналитического и численного расчёта.....	31
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33
ПРИЛОЖЕНИЯ.....	35
Приложение А – Код файла « <i>formGenerator.js</i> »	35
Приложение Б – Код файла « <i>meshGenerator.js</i> »	38
Приложение В – Код файла « <i>conditionsGenerator.js</i> ».....	42

Приложение Г – Код файла « <i>server.js</i> ».....	44
Приложение Д – Код файла « <i>run.js</i> »	46
Приложение Е – Код файла « <i>validation.js</i> »	48
Приложение Ж – Код файла « <i>visualize.py</i> »	53
Приложение И – Код файла « <i>public/index.html</i> »	60
Приложение К – Код файла « <i>public/styles.css</i> »	64
Приложение Л – Код файла « <i>Run.sh</i> »	72

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке используются следующие сокращения и условные обозначения:

Обозначение	Расшифровка
CFD	вычислительная гидродинамика (Computational Fluid Dynamics)
FVM	метод конечных объёмов (Finite Volume Method)
GAMG	геометро-алгебраический многосеточный метод решения систем линейных алгебраических уравнений
SIMPLE	полунявный метод для связи скорости и давления (Semi-Implicit Method for Pressure-Linked Equations)
САПР	система автоматизированного проектирования
ПО	программное обеспечение
ВМ	виртуальная машина
ОС	операционная система
HTML/CSS/JS	языки разметки, стилей и сценариев веб-страниц
p	кинематическое давление, $\text{м}^2/\text{с}^2$
Re	число Рейнольдса, безразмерная величина
S_{big}	размер большого участка расчётной геометрии, мм
S_{small}	размер малого участка расчётной геометрии, мм
L_{rect}	расстояние между центрами основных участков геометрии, мм
L_{pipe}	длина входного и выходного патрубков, мм
W	ширина входного и выходного патрубков, мм
U	скорость потока, м/с
ν	кинематическая вязкость воздуха, $\text{м}^2/\text{с}$

ВВЕДЕНИЕ

В современных инженерных задачах, связанных с аэродинамикой, вентиляцией и проектированием различных каналов и воздуховодов, моделирование движения воздушных потоков в областях сложной формы играет важную роль. Традиционные аналитические методы расчёта не всегда позволяют в полной мере учитывать особенности геометрии и характер распределения скорости и давления, а проведение натурных экспериментов может быть технически затруднено или экономически нецелесообразно. Поэтому для решения подобных задач широко применяются методы вычислительной гидродинамики и специализированные программные средства, среди которых особое место занимает свободно распространяемый пакет OpenFOAM, основанный на методе конечных объёмов.

Несмотря на широкие возможности, использование OpenFOAM связано с рядом сложностей. Подготовка расчёта, построение расчётной сетки, задание начальных и граничных условий, настройка решателя и запуск моделирования выполняются преимущественно посредством редактирования текстовых конфигурационных файлов и работы с командной строкой. Это повышает трудоёмкость подготовки расчёта и усложняет использование программного комплекса пользователями, не имеющими достаточного опыта работы с OpenFOAM.

Цель данной работы – выполнить расчёт течения воздуха через канал заданной формы и визуализировать полученные результаты. Для достижения поставленной цели необходимо сформировать расчётную геометрию и сетку, задать физические параметры воздуха и граничные условия, подготовить конфигурационные файлы OpenFOAM и обеспечить автоматизированный запуск расчёта. Конечным результатом является веб-приложение, позволяющее задавать геометрические параметры модели и скорость потока воздуха на входе, автоматически выполнять расчёт в OpenFOAM 9 и получать визуализацию полей скорости и давления, а также линий тока средствами ParaView. Расчётная геометрия варианта представлена ниже (Рисунок 1).

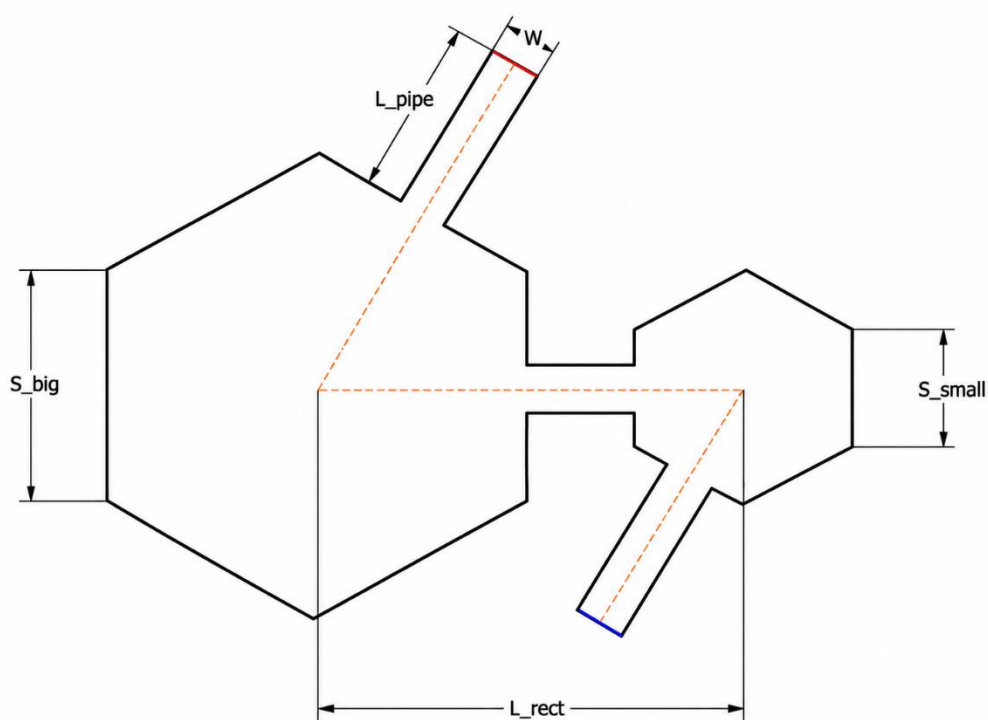


Рисунок 1 – Расчётная геометрия

Разрабатываемое решение позволяет задавать геометрические параметры канала и скорость воздушного потока через удобный веб-интерфейс. После ввода исходных данных автоматически формируется расчётная модель и запускается расчёт в OpenFOAM. Результаты обрабатываются в ParaView и представляются в виде полей скорости, давления и линий тока. Это позволяет упростить подготовку численного эксперимента и сделать работу с расчётной моделью более наглядной.

1 ОБЗОР И АНАЛИЗ НАУЧНО-ТЕХНИЧЕСКОЙ ЛИТЕРАТУРЫ

Для расчёта движения воздуха в каналах применяются методы вычислительной гидродинамики (CFD). С их помощью можно определить скорость и давление потока в разных участках расчётной области. Это особенно удобно для геометрии сложной формы, где получение аналитического решения затруднено.

Одним из программных комплексов для таких расчётов является OpenFOAM. Он основан на методе конечных объёмов и позволяет решать различные задачи механики жидкости и газа [1]. В состав OpenFOAM входят средства для построения сетки, задания свойств среды и граничных условий, а также численного решения задачи.

Возможность использования OpenFOAM для расчёта течения в каналах показана, например, в работе Смольянова и соавторов [2]. В ней рассматривалось течение электропроводной жидкости в канале. Результаты расчёта сравнивались с аналитическим решением и результатами, полученными в коммерческих программных пакетах. Авторы показали, что OpenFOAM может применяться в качестве расчётного инструмента для подобных задач.

При этом работа с OpenFOAM требует подготовки нескольких конфигурационных файлов. В них задаются сетка, физические свойства среды, начальные и граничные условия, а также параметры решателя. Запуск расчёта выполняется через отдельные утилиты и команды. Из-за этого даже для изменения нескольких исходных параметров пользователю приходится повторять ряд действий вручную.

Для решения этой проблемы существуют различные графические оболочки. В работе [3] описано приложение OpenFOAM_decompose_GUI, созданное на Python с использованием PyQt5. Через его интерфейс можно задавать параметры расчёта и граничные условия, работать со структурой проекта и запускать отдельные утилиты OpenFOAM. Такой пример

показывает, что часть операций по подготовке расчёта может быть автоматизирована.

Основы работы с OpenFOAM также рассмотрены в учебном пособии [4]. В нём приведены примеры построения сетки, настройки граничных условий, выбора схем дискретизации и решателей. Эти этапы используются и при подготовке расчётной модели в данной работе.

Для просмотра результатов расчёта используется ParaView [5]. Программа позволяет визуализировать полученные в OpenFOAM данные и анализировать распределение физических величин в расчётной области. В данной работе с помощью ParaView строятся поля скорости и давления, а также линии тока.

Рассмотренные источники показывают, что OpenFOAM подходит для расчёта движения воздуха в каналах, однако подготовка расчётного случая включает большое количество ручных операций. Поэтому в курсовом проекте часть этих действий автоматизируется. Пользователь задаёт исходные параметры через веб-интерфейс, после чего необходимые файлы формируются программой и выполняется расчёт.

2 АНАЛИТИЧЕСКИЙ РАСЧЕТ И МОДЕЛИРОВАНИЕ В OPENFOAM

2.1 Исходные данные и аналитический расчёт

В данном разделе выполняется предварительный аналитический расчёт параметров воздушного потока и проводится численное моделирование в OpenFOAM 9. Для расчёта используются значения параметров, заданные в разработанном приложении по умолчанию. Аналитический расчёт позволяет определить режим течения и получить оценочные значения скорости и перепада давления, которые в дальнейшем могут быть сопоставлены с результатами численного моделирования. Исходные данные приведены в таблице 1.

Таблица 1 – Параметры расчётной модели

Параметр	Значение
Размер большого участка S_{big}	20 мм
Размер малого участка S_{small}	10 мм
Расстояние между центрами L_{rect}	35 мм
Длина патрубков L_{pipe}	15 мм
Ширина канала и патрубков W	4 мм
Скорость воздуха на входе U	3 м/с
Кинематическая вязкость воздуха ν	$1,5 \cdot 10^{-5}$ м ² /с

Для определения режима течения рассчитывается число Рейнольдса. В качестве характерного размера принимается ширина канала W :

$$Re = \frac{U \cdot W}{\nu} \quad (1)$$

При $U = 3$ м/с, $W = 0,004$ м и $\nu = 1,5 \cdot 10^{-5}$ м²/с получаем:

$$Re = \frac{3 \cdot 0,004}{1,5 \cdot 10^{-5}} = 800.$$

Полученное значение $Re = 800$ меньше принятого предельного значения 2300, поэтому течение рассматривается как ламинарное. Это условие также используется в разработанном приложении при проверке введенных пользователем параметров.

При прохождении воздуха через широкую часть расчётной области скорость потока уменьшается. Ширина большого участка может быть приближённо определена по выражению:

$$W_{big} = S_{big} \cdot \sqrt{3} \quad (2)$$

При $S_{big} = 20\text{мм}$ получаем:

$$W_{big} = 20 \cdot \sqrt{3} \approx 34,6 \text{ мм.}$$

Среднюю скорость воздуха в широкой части области можно оценить из условия сохранения расхода:

$$U_{big} \approx U \frac{W}{W_{big}} \quad (3)$$

Тогда:

$$U_{big} = 3 \frac{4}{34,6} \approx 0,35 \text{ м/с}$$

Таким образом, при переходе из патрубка шириной 4 мм в широкую часть расчётной области средняя скорость потока должна существенно уменьшаться. При дальнейшем прохождении воздуха через узкий соединительный канал скорость снова возрастает.

Для ламинарного течения между параллельными стенками профиль скорости в установившемся режиме близок к параболическому. Максимальная скорость такого потока может быть оценена как:

$$U_{max} \approx 1,5U \quad (4)$$

Для принятой скорости $U = 3\text{м/с}$ получаем:

$$U_{max} \approx 1,5 \cdot 3 = 4,5 \text{ м/с.}$$

Полученное значение является приближённой аналитической оценкой. Реальное распределение скорости зависит от формы расчётной области, поворотов потока, расширений и сужений и определяется далее численным методом.

Для последующего сравнения с результатами OpenFOAM выполняется приближённая оценка перепада кинематического давления. Вязкие потери на

прямых узких участках могут быть оценены по зависимости для плоского ламинарного течения:

$$\Delta p_v = 12\nu \frac{LU}{W^2} \quad (5)$$

где Δp_v – перепад кинематического давления, $\text{м}^2/\text{с}^2$; L – суммарная длина узких прямых участков, м.

Длина соединительного канала определяется как:

$$L_{ch} = L_{rect} - \frac{(S_{big} + S_{small})\sqrt{3}}{2} \quad (6)$$

При исходных размерах:

$$L_{ch} = 35 - \frac{(20 + 10) \cdot \sqrt{3}}{2} \approx 9,02 \text{ мм.}$$

С учётом входного и выходного патрубков суммарная длина составляет:

$$L = 2L_{pipe} + L_{ch} = 2 \cdot 15 + 9,02 = 39,02 \text{ мм} = 0,03902 \text{ м.}$$

Тогда вязкая составляющая потерь равна:

$$\Delta p_v = 12 \cdot 1,5 \cdot 10^{-5} \cdot \frac{0,03902 \cdot 3}{0,004^2} \approx 1,32 \text{ м}^2/\text{с}^2.$$

Кроме потерь на трение, в расчётной области возникают местные потери, связанные с поворотами потока, расширениями и сужениями. Для предварительной оценки суммарный коэффициент местных сопротивлений принимается равным $\zeta \approx 2,0$.

Тогда:

$$\Delta p_m = \zeta \frac{U^2}{2} \quad (7)$$

$$\Delta p_m = 2,0 \frac{3^2}{2} = 9 \text{ м}^2/\text{с}^2.$$

Оценочный суммарный перепад кинематического давления составляет:

$$\Delta p = \Delta p_v + \Delta p_m \approx 1,32 + 9 \approx 10,32 \text{ м}^2/\text{с}^2. \quad (8)$$

Следовательно, в качестве опорных значений для последующего сравнения принимаются: число Рейнольдса $Re = 800$, средняя скорость в широкой части области около $0,35 \text{ м/с}$, максимальная скорость в узком участке

около 4,5 м/с и оценочный перепад кинематического давления около $10,32 \text{ м}^2/\text{с}^2$. Последние два значения являются приближёнными и должны уточняться по результатам численного моделирования.

2.2 Построение расчётной сетки

Для проверки предварительных оценок расчёт выполняется в OpenFOAM 9. Расчётная область строится утилитой blockMesh. Геометрия задаётся в плоскости XY и имеет одну ячейку по оси Z. Передняя и задняя грани задаются типом empty, что позволяет рассматривать задачу как двумерную.

Перед выполнением расчёта качество созданной сетки проверяется утилитой checkMesh. Стенки канала объединены в группу walls, торец входного патрубка относится к границе inlet, а выходного – к outlet.

Построенная расчётная сетка представлена на рисунке 2.

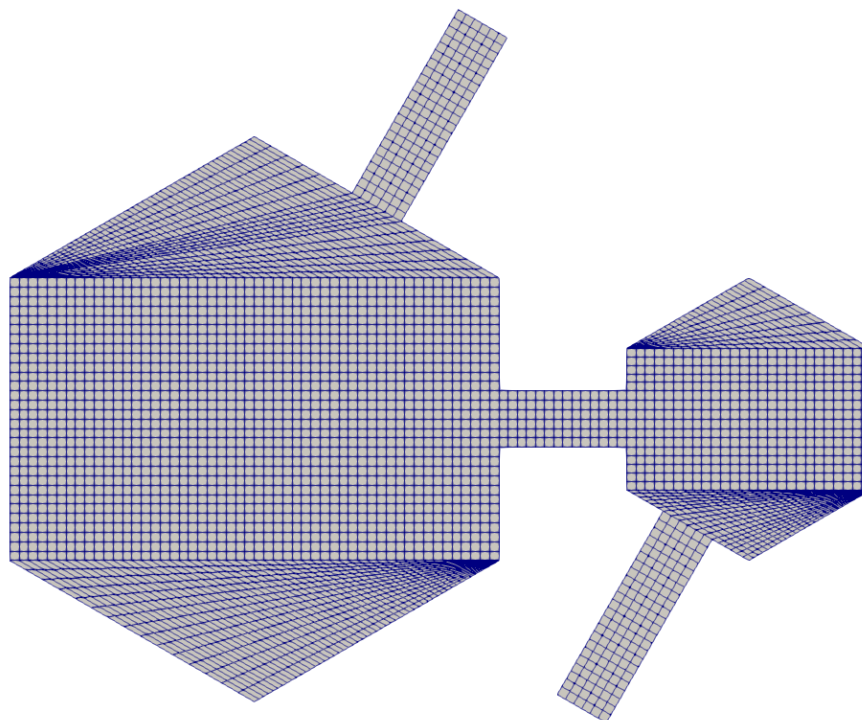


Рисунок 2 – Расчётная сетка, построенная утилитой blockMesh

2.3 Настройка расчётной модели в OpenFOAM

После формирования и проверки расчётной сетки запускается решатель simpleFoam. По завершении расчёта формируются поля скорости и

кинематического давления, которые далее используются для визуализации и анализа результатов в ParaView.

Кинематическая вязкость воздуха принимается равной:

$$\nu = 1,5 \cdot 10^{-5} \text{ м}^2/\text{с}. \quad (9)$$

Для расчётной области задаются следующие граничные условия. На входной границе inlet устанавливается постоянная скорость воздушного потока $U = 3 \text{ м/с}$, направленная внутрь расчётной области, а для давления задаётся нулевой градиент. На выходной границе outlet используется нулевой градиент скорости и фиксированное значение кинематического давления $p = 0$. На стенках walls применяется условие прилипания noSlip. Передняя и задняя грани frontAndBack имеют тип empty, что соответствует двумерной постановке задачи.

Расчёт выполняется в стационарной постановке. Для связи полей скорости и давления применяется алгоритм SIMPLE. Уравнение давления решается методом GAMG, а для поля скорости используется smoothSolver. В настройках алгоритма задаются два корректора неортогональности сетки. Критерий сходимости для давления и скорости принимается равным 10^{-4} .

Запуск расчёта выполняется последовательно командами blockMesh, checkMesh и simpleFoam. По завершении решения получаются поля скорости и кинематического давления, которые далее обрабатываются средствами ParaView.

2.4 Результаты численного моделирования

После завершения расчёта результаты визуализируются в ParaView. Для анализа используются поле модуля скорости $|U|$, продольная составляющая скорости U_x , поле кинематического давления p и линии тока.

На рисунке 3 представлено распределение модуля скорости в расчётной области.

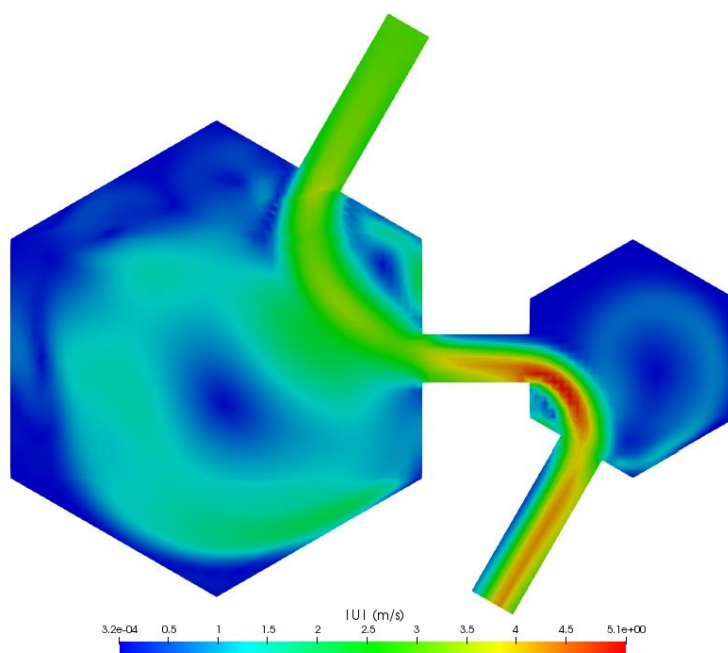


Рисунок 3 – Поле модуля скорости воздушного потока

По распределению скорости видно, что после поступления воздуха через входной патрубок поток попадает в широкую часть расчётной области, где его скорость уменьшается. При прохождении через узкие участки скорость снова возрастает. Такое распределение соответствует результатам предварительной аналитической оценки, согласно которой расширение потока приводит к уменьшению средней скорости.

На рисунке 4 приведено распределение кинематического давления.

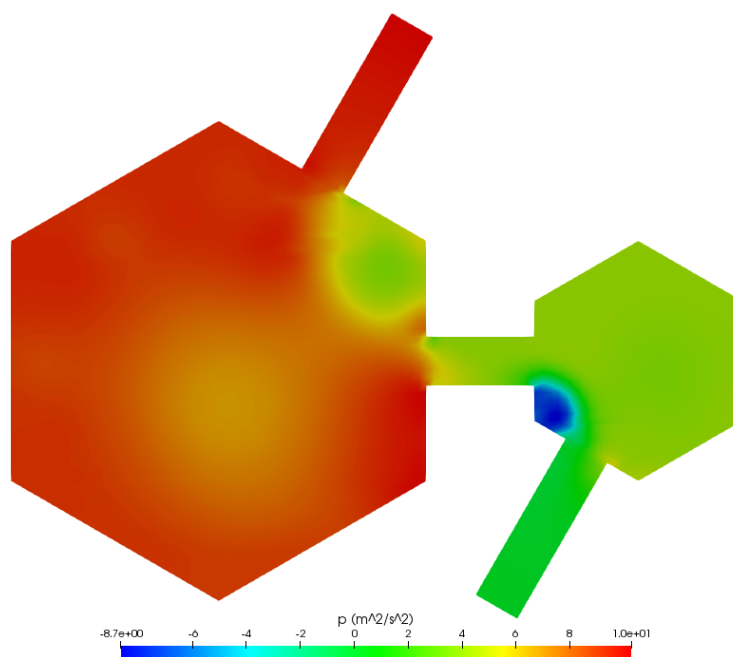


Рисунок 4 – Поле кинематического давления

Кинематическое давление в целом уменьшается по направлению от входной границы к выходной. Наиболее заметное изменение давления связано с участками поворота, расширения и сужения потока. Значение давления на выходной границе принимается равным нулю в соответствии с заданным граничным условием.

Характер движения воздуха дополнительно оценивается по линиям тока, представленным на рисунке 5.

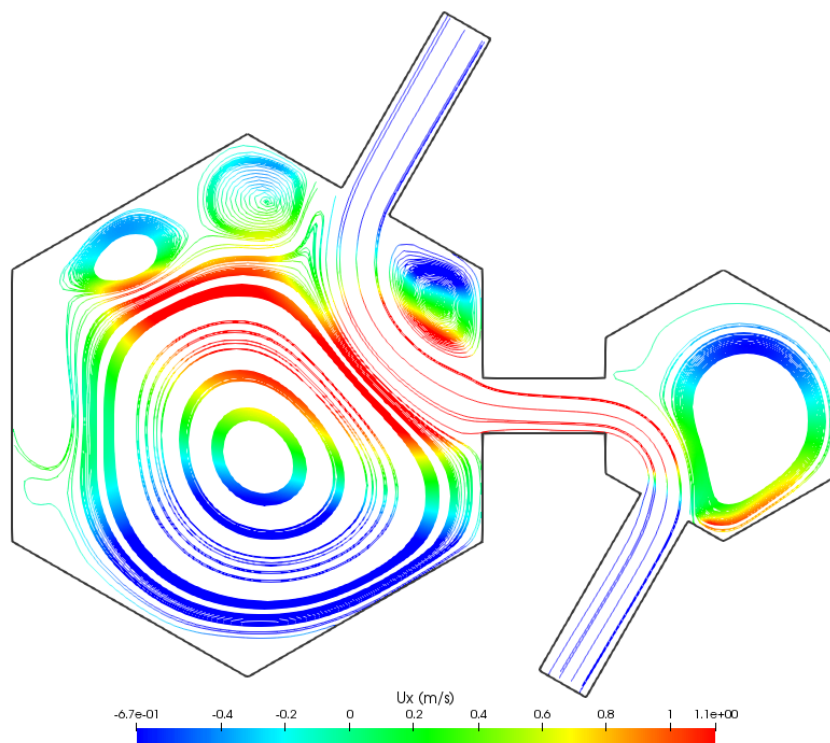


Рисунок 5 – Линии тока воздушного потока

Линии тока позволяют проследить движение воздуха от входного патрубка к выходному и определить участки, в которых направление потока изменяется под влиянием геометрии канала. В широких областях возможно образование зон с пониженной скоростью и локального возвратного движения.

3 АНАЛИЗ И ПОДГОТОВКА ГЕОМЕТРИИ

3.1 Исходные данные

Задачей курсового проекта является разработка приложения для расчёта течения воздуха через канал заданной формы средствами OpenFOAM и ParaView. Пользователь может изменять основные геометрические параметры модели и скорость воздушного потока на входе.

Расчётная область состоит из двух правильных шестиугольников различного размера, соединённых прямым каналом шириной W . К большому шестиугольнику под углом присоединяется входной патрубок, а к малому – выходной. Ширина обоих патрубков равна W , а их длина определяется параметром L_{pipe} . Расстояние между центрами шестиугольников задаётся параметром L_{rect} .

Основными параметрами геометрии являются S_{big} , S_{small} , L_{rect} , L_{pipe} и W . Кроме геометрических размеров пользователь задаёт скорость воздушного потока U на входе. Расчётная геометрия с обозначением основных параметров представлена на рисунке 1.

В качестве расчётной среды используется воздух. Режим течения определяется по числу Рейнольдса, рассчитанному по формуле (1). При параметрах по умолчанию $Re = 800$, поэтому течение рассматривается как ламинарное.

3.2 Подготовка геометрии

Перед построением расчётной сетки определяются координаты основных точек геометрии. Они рассчитываются на основе параметров, введённых пользователем, и далее используются при формировании расчётной области.

Для правильного шестиугольника со стороной S апофема определяется выражением:

$$a = \frac{S\sqrt{3}}{2} \quad (10)$$

Для большого и малого шестиугольников при параметрах по умолчанию значения апофем составляют соответственно 17,32 мм и 8,66 мм.

Центры шестиугольников располагаются на расстоянии L_{rect} друг от друга. Длина соединительного канала определяется с учётом апофем большого и малого шестиугольников. При $L_{rect} = 35$ мм она составляет приблизительно 9,02 мм.

Полученное значение должно быть положительным, поэтому перед формированием модели выполняется проверка корректности введённых геометрических параметров.

Входной и выходной патрубки присоединяются к наклонным граням шестиугольников. Их ширина равна W , а длина определяется параметром L_{pipe} .

Для использования модели в OpenFOAM двумерная геометрия продлевается по оси Z на одну ячейку. Передняя и задняя грани задаются типом `empty`, что позволяет выполнять расчёт в псевдодвумерной постановке.

4 НАПИСАНИЕ КОДА СОЗДАНИЯ МОДЕЛИ

4.1 Создание топологии

Геометрия расчётной области формируется в файле `formGenerator.js`. Класс `VerticesGenerator` получает заданные пользователем параметры и рассчитывает координаты вершин большого и малого шестиугольников, соединительного канала, а также входного и выходного патрубков. Для псевдодвумерной постановки каждая точка создаётся в двух плоскостях по оси *Z*. Полученные координаты далее используются при формировании словаря `blockMeshDict`.

Листинг 1 – Фрагмент файла `formGenerator.js`

```
// Координаты геометрии. Размеры задаются в миллиметрах.
const SQRT3_2 = Math.sqrt(3) / 2;
class VerticesGenerator {
  constructor(sBig, sSmall, lRect, lPipe, w) {
    this.sBig = sBig;
    this.sSmall = sSmall;
    this.lRect = lRect;
    this.lPipe = lPipe;
    this.w = w;
    this.coords = {};
    this.names = [];
  }
  generateVertices() {
    const { sBig, sSmall, lRect, lPipe, w } = this;
    const aBig = sBig * SQRT3_2;
    const aSmall = sSmall * SQRT3_2;
    const hBig = sBig / 2;
    const hSmall = sSmall / 2;
    const w2 = w / 2;
    const depth = 1;
    const cxBig = aBig;
    const cxSmall = aBig + lRect;
    const channelLen = lRect - aBig - aSmall;
```

4.2 Генерация файла расчётной сетки

Файл `meshGenerator.js` содержит класс `MeshGenerator`, формирующий `system/blockMeshDict`. Расчётная область разбивается на структурированные блоки, для которых автоматически определяются количество ячеек и номера вершин. По ширине канала задаётся шесть ячеек, а по оси *Z* – одна. Границы модели разделяются на `inlet`, `outlet`, `walls` и `frontAndBack`.

Листинг 2 – Фрагмент файла meshGenerator.js

```
// Размер ячейки связываем с шириной канала
const hc = Math.max(vg.w / 6, 1e-9);
const r = (x, m) => Math.max(m, Math.round(x / hc));
const nW = r(vg.w, 6);
const nXbig = r(2 * vg.aBig, 6);
const nXsm = r(2 * vg.aSmall, 6);
const nChanL = r(vg.channelLen, 8);
const nPipeL = r(vg.lPipe, 8);
const blocks = [];
const B = (q, c, tag) => blocks.push({ q, c, tag });
B(
  ['B_CH_B', 'S_CH_B', 'S_CH_T', 'B_CH_T'],
  [nChanL, nW],
  'channel'
);
B(
  ['B_IA', 'B_IB', 'B_FB', 'B_FA'],
  [nW, nPipeL],
  'inletPipe'
);
B(
  ['S_OB', 'S_OA', 'S_FA', 'S_FB'],
  [nW, nPipeL],
  'outletPipe'
);
```

4.3 Генерация начальных и граничных условий

Начальные и граничные условия формируются в файле conditionsGenerator.js. Программно создаются поля скорости $0/U$ и кинематического давления $0/p$. На входе задаётся постоянная скорость, направленная внутрь расчётной области, на выходе – нулевой градиент скорости и фиксированное давление $p = 0$. На стенках применяется условие noSlip, а грани frontAndBack имеют тип empty.

Листинг 3 – Фрагмент файла conditionsGenerator.js

```
class ConditionsGenerator {
  exportU(speed, casePath) {
    const ux = (-0.5 * speed).toFixed(6);
    const uy = (-SQRT3_2 * speed).toFixed(6);
    const text = `${header('volVectorField', 'U')}`
  }
  dimensions [0 1 -1 0 0 0 0];
  internalField uniform (0 0 0);
  boundaryField
  {
    inlet
    {
```

Продолжение листинга 3

```
    type      fixedValue;  
    value     uniform ($ {ux} $ {uy} 0);  
  }  
  outlet  
  {  
    type      zeroGradient;  
  }  
  walls  
  {  
    type      noSlip;  
  }  
  frontAndBack  
  {  
    type      empty;  
  }  
};
```

4.4 Настройка решателя

Для расчёта используется simpleFoam пакета OpenFOAM 9. Расчёт выполняется в стационарной ламинарной постановке с использованием алгоритма SIMPLE. Для давления применяется решатель GAMG, для скорости – smoothSolver. Критерий сходимости для полей давления и скорости установлен равным 10^{-4} .

Основные параметры решателя задаются в файлах controlDict, fvSchemes и fvSolution.

Листинг 4 – Фрагмент файла fvSolution

```
solvers  
{  
  p  
  {  
    solver      GAMG;  
    smoother     GaussSeidel;  
    tolerance    1e-08;  
    relTol       0.01;  
  }  
  U  
  {  
    solver      smoothSolver;  
    smoother     symGaussSeidel;  
    tolerance    1e-08;  
    relTol       0.05;  
  }  
}
```

Продолжение листинга 4

```
SIMPLE
{
  nNonOrthogonalCorrectors 2;

  residualControl
  {
    p      1e-4;
    U      1e-4;
  }
}
relaxationFactors
{
  fields
  {
    p      0.2;
  }
  equations
  {
    U      0.4;
  }
}
```

5 РАЗРАБОТКА ПРИЛОЖЕНИЯ

5.1 Архитектура приложения

Разработанное приложение построено по клиент-серверной архитектуре. Клиентская часть представляет собой веб-страницу на HTML, CSS и JavaScript, через которую пользователь задаёт геометрические параметры модели и скорость воздушного потока. Серверная часть реализована на платформе Node.js с использованием фреймворка Express.

Введённые пользователем данные передаются на сервер, где `server.js` принимает запрос и запускает управляющий модуль `run.js`. Он вызывает `formGenerator.js`, `meshGenerator.js` и `conditionsGenerator.js`, формирующие геометрию, расчётную сетку и граничные условия.

Далее выполняются построение сетки, её проверка и численный расчёт средствами OpenFOAM. Для этого используются утилиты `blockMesh`, `checkMesh` и решатель `simpleFoam`. Полученные данные обрабатываются в ParaView, а сформированные изображения полей скорости, давления и линий тока отображаются в браузере.

Общая архитектура разработанного приложения представлена на рисунке 6.

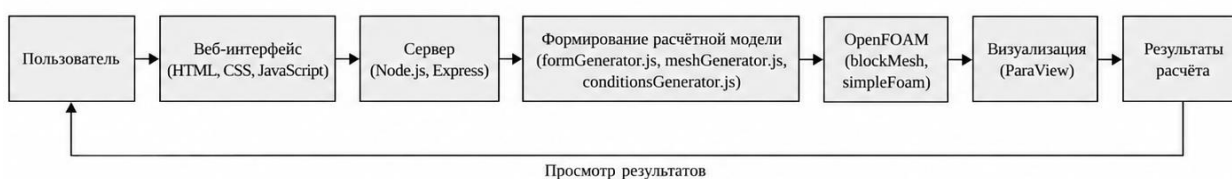


Рисунок 6 – Архитектура разработанного приложения

5.2 Разработка клиентской части

Клиентская часть приложения расположена в каталоге `public` и включает файлы `index.html`, `styles.css` и `validation.js`. Пользователь может изменять параметры S_{big} , S_{small} , L_{rect} , L_{pipe} , W и скорость воздушного потока U . Перед отправкой данных выполняется их

проверка: контролируется положительность значений, соотношение размеров и условие ламинарного течения.

Интерфейс разработанного приложения представлен на рисунке 7.

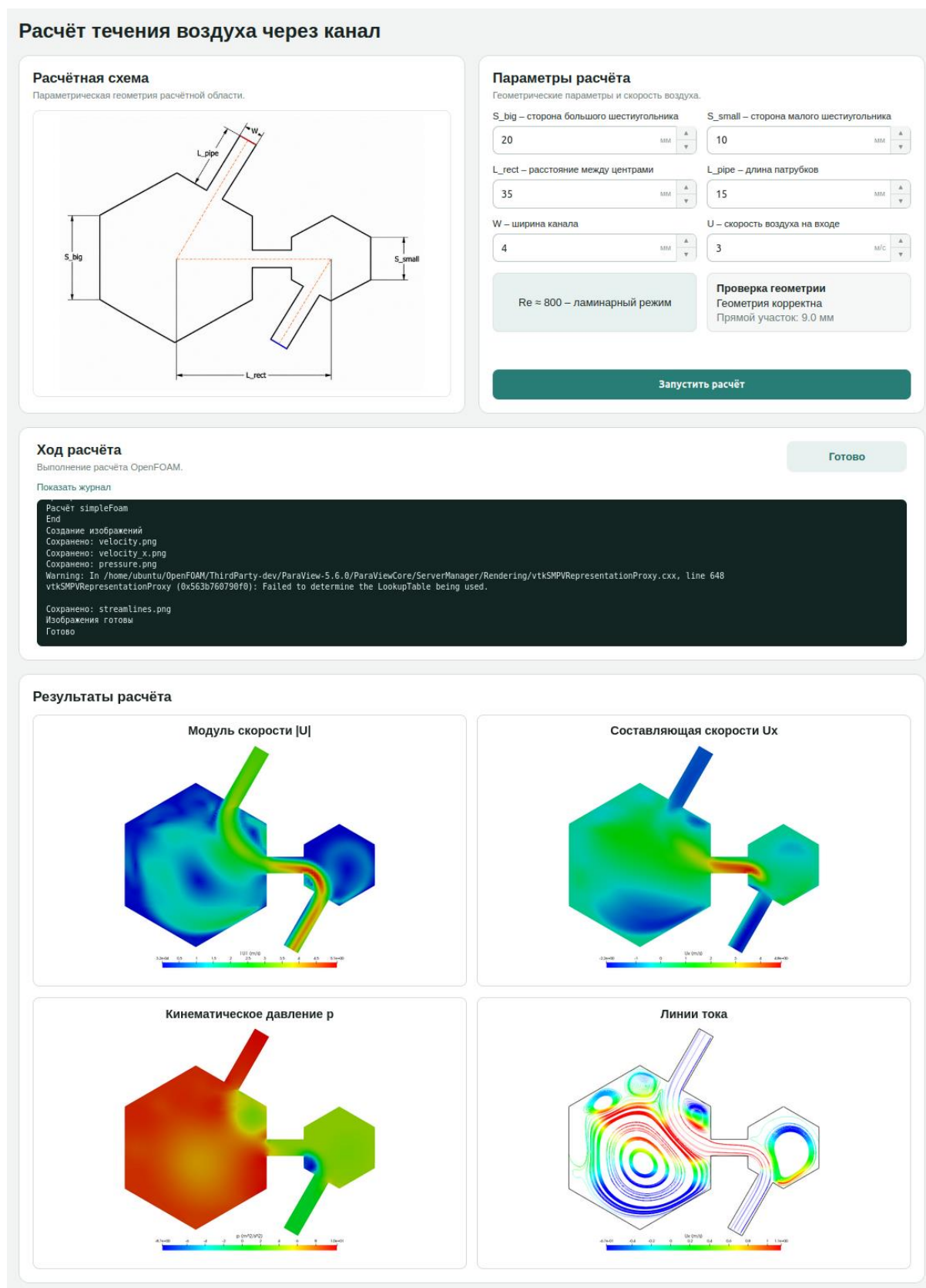


Рисунок 7 – Интерфейс разработанного приложения

На странице отображаются расчётная схема, поля ввода параметров, информация о корректности геометрии, кнопка запуска расчёта, журнал выполнения и результаты моделирования. Проверка введённых пользователем значений реализована в файле validation.js. Фрагмент данного файла приведён в листинге 5.

Листинг 5 – Фрагмент файла validation.js

```
const AIR_NU = 1.5e-5;
const MAX_RE = 2300;
const SQRT3_2 = Math.sqrt(3) / 2;
function reynolds(p) {
    return p.speed * (p.w / 1000) / AIR_NU;
}
function checkGeometry(p) {
    if ([p.sBig, p.sSmall, p.lRect, p.lPipe, p.w]
        .some(v => !Number.isFinite(v) || v <= 0)) {
        return { ok: false, text: 'Все размеры должны быть больше нуля' };
    }
    if (p.sSmall >= p.sBig) {
        return { ok: false, text: 'S_small должен быть меньше S_big' };
    }
    if (p.w >= p.sSmall) {
        return { ok: false, text: 'W должен быть меньше S_small' };
    }
    const aBig = p.sBig * SQRT3_2;
    const aSmall = p.sSmall * SQRT3_2;
    const length = p.lRect - aBig - aSmall;
    if (length <= 0) {
        return {
            ok: false,
            text: `Увеличьте L_rect минимум до ${ (aBig + aSmall).toFixed(1)} мм`
        };
    }
    return { ok: true, length };
}
```

5.3 Разработка серверной части

Серверная часть реализована в файле server.js на платформе Node.js с использованием Express. Для запуска расчёта используется маршрут /calculate, принимающий параметры модели в формате JSON. После получения запроса сервер запускает файл run.js как отдельный процесс.

Файл run.js выполняет роль управляющего модуля. Он повторно проверяет входные значения, запускает генераторы геометрии, сетки и

граничных условий, после чего выполняет сценарий расчёта OpenFOAM. По завершении расчёта запускается скрипт visualize.py, формирующий изображения полей скорости, давления и линий тока.

Для получения информации о текущем состоянии расчёта используется маршрут /status. Клиент периодически обращается к нему и отображает текущий этап выполнения и журнал работы.

Листинг 6 – Фрагмент файла server.js

```
const express = require('express');
const { spawn } = require('child_process');
const path = require('path');
const app = express();
app.use(express.json());
app.use(express.static('public'));
let currentStatus = 'idle';
let currentLog = "";
app.post('/calculate', (req, res) => {
  if (currentStatus === 'running') {
    return res.status(409).json({
      success: false,
      error: 'Расчёт уже выполняется'
    });
  }
  const { sBig, sSmall, lRect, lPipe, w, speed } = req.body;
  const args = [sBig, sSmall, lRect, lPipe, w, speed].map(String);
  currentStatus = 'running';
  currentLog = "";
  const child = spawn(
    'node',
    [path.join(__dirname, 'run.js'), ...args],
    { cwd: __dirname }
  );
  child.stdout.on('data', data => {
    currentLog += data.toString();
  });
  child.on('close', code => {
    currentStatus = code === 0 ? 'done' : 'error';
  });
  res.json({ success: true, message: 'Расчёт запущен' });
});
app.get('/status', (req, res) => {
  res.json({
    status: currentStatus,
    log: currentLog
  });
});
```

6 ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ И СРАВНЕНИЕ РЕЗУЛЬТАТОВ

6.1 Тестирование приложения

Тестирование приложения выполнялось в виртуальной машине Ubuntu 20.04 с установленным OpenFOAM 9. После запуска сервера веб-интерфейс становится доступен в браузере по адресу <http://localhost:3000>.

Для проверки механизма валидации были специально введены некорректные значения параметров: $L_{rect} = 5$ мм и $U = 15,51$ м/с. При таких исходных данных невозможно корректное построение геометрии, а число Рейнольдса превышает допустимое для принятого ламинарного режима значение. Приложение выводит соответствующие сообщения и блокирует запуск расчёта.

Результат проверки представлен на рисунке 8.

The screenshot shows a web application interface titled "Параметры расчёта" (Calculation Parameters). Below the title is a subtitle "Геометрические параметры и скорость воздуха." (Geometric parameters and air velocity). The interface contains several input fields with labels and units:

- S_big** – сторона большого шестиугольника (Side of the large hexagon): input field with value 20, unit мм (mm).
- S_small** – сторона малого шестиугольника (Side of the small hexagon): input field with value 10, unit мм (mm).
- L_rect** – расстояние между центрами (Distance between centers): input field with value 5, unit мм (mm).
- L_pipe** – длина патрубков (Pipe length): input field with value 15, unit мм (mm).
- W** – ширина канала (Channel width): input field with value 4, unit мм (mm).
- U** – скорость воздуха на входе (Air velocity at inlet): input field with value 15,51, unit м/с (m/s).

Below the input fields, there are two informational boxes:

- A light blue box on the left states: "Re ≈ 4136 – вне ламинарного диапазона" (Re ≈ 4136 – outside the laminar range).
- A light red box on the right states: "Проверка геометрии" (Geometry check) and "Увеличьте L_rect минимум до 26.0 мм" (Increase L_rect at least to 26.0 mm).

At the bottom of the interface is a large teal button labeled "Запустить расчёт" (Run calculation).

Рисунок 8 – Проверка некорректных входных данных

После проверки валидации был выполнен расчёт с параметрами по умолчанию: $S_{big} = 20$ мм, $S_{small} = 10$ мм, $L_{rect} = 35$ мм, $L_{pipe} = 15$ мм, $W = 4$ мм и $U = 3$ м/с. Для данного набора параметров число Рейнольдса составляет 800, что соответствует принятому ламинарному режиму течения.

При запуске расчёта приложение автоматически формирует расчётный случай, создаёт и проверяет сетку, запускает решатель simpleFoam, а затем выполняет обработку результатов средствами ParaView.

По завершении расчёта в веб-интерфейсе отображаются поле модуля скорости, продольная составляющая скорости U_x , поле кинематического давления и линии тока. Результаты численного моделирования представлены на рисунке 9.

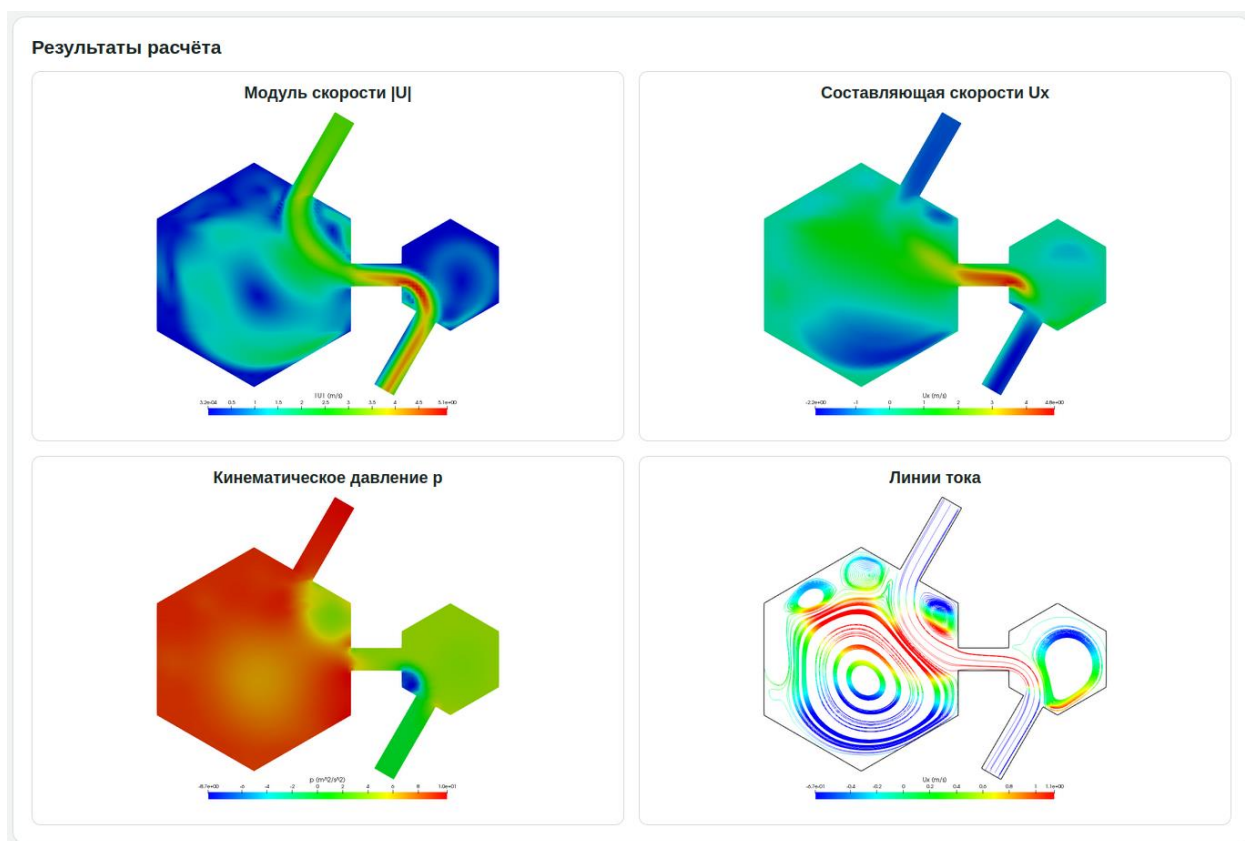


Рисунок 9 – Результаты численного моделирования в приложении

Полученная картина течения соответствует ожидаемому поведению воздушного потока. В расширенных участках расчётной области скорость уменьшается, а при прохождении через узкий соединительный канал возрастает. Поле кинематического давления изменяется по направлению

движения воздуха. Линии тока показывают основной путь движения потока и наличие циркуляционных зон в расширенных участках модели.

6.2 Сравнение аналитического и численного расчёта

Для оценки корректности разработанной модели результаты численного моделирования сопоставляются с аналитическими значениями, полученными ранее. Сравнение выполняется по числу Рейнольдса, максимальной скорости потока и перепаду кинематического давления.

Таблица 2 – Сравнение результатов аналитического и численного расчёта

Величина	Аналитический расчет	OpenFOAM	Расхождение, %
Число Рейнольдса Re	800	800	0
Макс. скорость потока в соединительном канале $ U $, м/с	4,5	4,76	5,67
Перепад кинематического давления Δp , m^2/c^2	10,32	10,21	1,1

Полученные результаты показывают хорошее соответствие аналитического и численного расчётов. Число Рейнольдса в обоих случаях составляет 800. Максимальная скорость в прямом соединительном канале по результатам OpenFOAM составляет 4,76 м/с при аналитической оценке 4,5 м/с. Расхождение составляет 5,67 %.

Перепад кинематического давления по результатам численного моделирования составляет 10,21 m^2/c^2 при аналитическом значении 10,32 m^2/c^2 . Расхождение составляет 1,10 %.

Таким образом, результаты численного моделирования в OpenFOAM согласуются с предварительными аналитическими оценками и подтверждают корректность выбранной расчётной модели.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были завершены основные этапы создания приложения для моделирования течения воздуха через канал заданной формы.

Выполнен анализ исходных данных, подготовлена параметрическая геометрия, сформированы расчётная сетка, начальные и граничные условия. Расчёт реализован в OpenFOAM 9 в стационарной ламинарной постановке.

Разработаны клиентская и серверная части веб-приложения, а также автоматизированы формирование расчётной модели, запуск OpenFOAM и визуализация результатов в ParaView. Сравнение аналитических и численных результатов показало их хорошее соответствие.

Поставленная цель достигнута: приложение позволяет задавать параметры модели и скорость потока, автоматически выполнять расчёт и получать визуализацию полей скорости, давления и линий тока.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. OpenFOAM Foundation. OpenFOAM: офиц. сайт. — URL: <https://openfoam.org/> (Дата обращения: 24.02.2026).
2. Верификация пакетов для расчета течений жидкости в канале при действии внешнего магнитного поля / И. А. Смольянов, Е. И. Шмаков, Э. Бааке, М. Гульельме // Вычислительная механика сплошных сред. — 2021. — Т. 14, № 3. — С. 322-332. — DOI 10.7242/1999-6691/2021.14.3.27. — EDN FHUMOM.
3. Читалов, Д. И. Разработка графической оболочки для параллельных расчетов на базе платформы OpenFOAM / Д. И. Читалов // Программные продукты и системы. — 2019. — № 3. — С. 433–443. — DOI: 10.15827/0236-235X.127.433-443. — EDN VRIIQG.
4. Введение в компьютерное моделирование гидродинамических процессов в программном комплексе OpenFOAM / А. Н. Нуриев, О. Н. Зайцева, А. М. Камалутдинов, О. С. Жучкова. — Казань: Издательство Казанского университета, 2022. — 84 с. — EDN RSHCTU.
5. ParaView: An End-User Tool for Large Data Visualization: офиц. сайт. — URL: <https://www.paraview.org/> (дата обращения: 15.03.2026).
6. Node.js: документация платформы. — URL: <https://nodejs.org/docs/> (дата обращения: 15.03.2026).
7. Express — веб-фреймворк для Node.js: офиц. сайт. — URL: <https://expressjs.com/> (дата обращения: 15.03.2026).
8. Greenshields, C. OpenFOAM v9 User Guide / C. Greenshields. — London : The OpenFOAM Foundation, 2021. — URL: <https://doc.cfd.direct/openfoam/user-guide-v9/> (дата обращения: 14.09.2026).
9. Ubuntu. The open source operating system for latest desktops: сайт / Canonical Ltd. — London, 2004— . — URL: <https://ubuntu.com/download/desktop> (дата обращения: 25.02.2026).

10. Versteeg, H. K. An Introduction to Computational Fluid Dynamics: The Finite Volume Method / H. K. Versteeg, W. Malalasekera. – 2nd ed. – Harlow: Pearson Education, 2007. – 520 c.

ПРИЛОЖЕНИЯ

Приложение А – Код файла «formGenerator.js»

```
// Координаты геометрии. Размеры задаются в миллиметрах.
const Sqrt3_2 = Math.sqrt(3) / 2;
class VerticesGenerator {
  constructor(sBig, sSmall, lRect, lPipe, w) {
    this.sBig = sBig;
    this.sSmall = sSmall;
    this.lRect = lRect;
    this.lPipe = lPipe;
    this.w = w;
    this.coords = {};
    this.names = [];
  }
  _add(name, x, y) {
    this.coords[name] = { x, y };
    this.names.push(name);
  }
  generateVertices() {
    const { sBig, sSmall, lRect, lPipe, w } = this;
    const aBig = sBig * Sqrt3_2;
    const aSmall = sSmall * Sqrt3_2;
    const hBig = sBig / 2;
    const hSmall = sSmall / 2;
    const w2 = w / 2;
    const depth = 1; // толщина слоя по Z, мм
    const cxBig = aBig;
    const cxSmall = aBig + lRect;
    const channelLen = lRect - aBig - aSmall;
    // Большой шестиугольник
    this._add('B_TOP', cxBig, sBig);
    this._add('B_UR', cxBig + aBig, hBig);
    this._add('B_LR', cxBig + aBig, -hBig);
    this._add('B_BOT', cxBig, -sBig);
    this._add('B_LL', cxBig - aBig, -hBig);
    this._add('B_UL', cxBig - aBig, hBig);
    this._add('B_C', cxBig, 0);
    // Окно канала на правой вертикальной грани (x = cxBig + aBig)
    this._add('B_CH_T', cxBig + aBig, w2);
    this._add('B_CH_B', cxBig + aBig, -w2);
    // Опорные точки на ЛЕВОЙ вертикальной грани (x = cxBig - aBig = 0)
    // на тех же высотах ±w2 — для горизонтальных полос сетки.
    this._add('B_LE_T', cxBig - aBig, w2);
    this._add('B_LE_B', cxBig - aBig, -w2);
    // Устье входного патрубка на верхней правой наклонной грани B_TOP->B_UR.
    // Единичное направление вдоль грани (TOP->UR) = (sqrt3/2, -1/2),
    // внешняя нормаль грани = (1/2, sqrt3/2).
    {
      const ex = Sqrt3_2, ey = -0.5; // вдоль грани
      const nx = 0.5, ny = Sqrt3_2; // внешняя нормаль
    }
  }
}
```

```

const mx = cxBig + aBig / 2; // середина грани
const my = (sBig + hBig) / 2; // = 3*sBig/4
const iax = mx - w2 * ex, iay = my - w2 * ey; // ближе к B_TOP
const ibx = mx + w2 * ex, iby = my + w2 * ey; // ближе к B_UR
this._add('B_IA', iax, iay);
this._add('B_IB', ibx, iby);
this._add('B_FA', iax + lPipe * nx, iay + lPipe * ny);
this._add('B_FB', ibx + lPipe * nx, iby + lPipe * ny);
}
// Малый шестиугольник
this._add('S_TOP', cxSmall, sSmall);
this._add('S_UR', cxSmall + aSmall, hSmall);
this._add('S_LR', cxSmall + aSmall, -hSmall);
this._add('S_BOT', cxSmall, -sSmall);
this._add('S_LL', cxSmall - aSmall, -hSmall);
this._add('S_UL', cxSmall - aSmall, hSmall);
this._add('S_C', cxSmall, 0);
// Окно канала на левой вертикальной грани малого hex (x = cxSmall - aSmall)
this._add('S_CH_T', cxSmall - aSmall, w2);
this._add('S_CH_B', cxSmall - aSmall, -w2);
// Опорные точки на ПРАВОЙ вертикальной грани малого hex (x = cxSmall +
aSmall)
// на высотах ±w2 — для горизонтальных полос сетки.
this._add('S_RE_T', cxSmall + aSmall, w2);
this._add('S_RE_B', cxSmall + aSmall, -w2);
// Устье выходного патрубка на нижней левой наклонной грани S_LL->S_BOT.
// Направление вдоль грани (LL->BOT) = (sqrt3/2, -1/2),
// внешняя нормаль = (-1/2, -sqrt3/2).
{
const dx = SQRT3_2, dy = -0.5; // вдоль грани LL->BOT
const nx = -0.5, ny = -SQRT3_2; // внешняя нормаль
const mx = cxSmall - aSmall / 2; // середина грани
const my = -(hSmall + sSmall) / 2; // = -3*sSmall/4
const oax = mx - w2 * dx, oay = my - w2 * dy; // ближе к S_LL
const obx = mx + w2 * dx, oby = my + w2 * dy; // ближе к S_BOT
this._add('S_OA', oax, oay);
this._add('S_OB', obx, oby);
this._add('S_FA', oax + lPipe * nx, oay + lPipe * ny);
this._add('S_FB', obx + lPipe * nx, oby + lPipe * ny);
}
// Эти значения используются при построении сетки
this.aBig = aBig;
this.aSmall = aSmall;
this.hBig = hBig;
this.hSmall = hSmall;
this.w2 = w2;
this.depth = depth;
this.cxBig = cxBig;
this.cxSmall = cxSmall;
this.channelLen = channelLen;
return this;
}

```

```

// Индекс 2D-точки по имени
pointIndex(name) {
  const i = this.names.indexOf(name);
  if (i < 0) throw new Error('Неизвестная точка: ${name}');
  return i;
}
// Для каждой точки создаются две вершины: z = 0 и z = 1 мм.
vBack(name) { return 2 * this.pointIndex(name); }
vFront(name) { return 2 * this.pointIndex(name) + 1; }
distance(a, b) {
  const p = this.coords[a], q = this.coords[b];
  return Math.hypot(p.x - q.x, p.y - q.y);
}
pointsToString() {
  const depth = this.depth;
  const f = (v) => v.toFixed(6);
  return this.names
    .map((name, i) => {
      const { x, y } = this.coords[name];
      return ` (${f(x)} ${f(y)} 0.000000) // ${2 * i} ${name}\n` +
        ` (${f(x)} ${f(y)} ${f(depth)}) // ${2 * i + 1} ${name}`;
    })
    .join('\n');
}
}
module.exports = { VerticesGenerator };

```

```

const fs = require('fs');
const path = require('path');
// Генерирует system/blockMeshDict для текущих размеров геометрии
class MeshGenerator {
  exportBlockMeshDict(vg, casePath) {
    // К каждой двумерной точке относятся задняя и передняя вершины по оси Z
    const vm = (n) => vg.vBack(n);
    const vp = (n) => vg.vFront(n);
    const idx = (n) => vg.pointIndex(n);
    // Размер ячейки связываем с шириной канала
    // Шесть ячеек по ширине дают более плавное распределение полей
    const hc = Math.max(vg.w / 6, 1e-9);
    // Количество ячеек округляется по выбранному размеру, но не опускается ниже
    минимума
    const r = (x, m) => Math.max(m, Math.round(x / hc));
    const nW = r(vg.w, 6);
    const nXbig = r(2 * vg.aBig, 6);
    const nXsm = r(2 * vg.aSmall, 6);
    const nYbig = r(vg.hBig - vg.w2, 3);
    const nYsm = r(vg.hSmall - vg.w2, 3);
    const nCapB = r(vg.sBig - vg.hBig, 3);
    const nCapS = r(vg.sSmall - vg.hSmall, 3);
    const nChanL = r(vg.channelLen, 8);
    const nPipeL = r(vg.lPipe, 8);
    // Каждый элемент массива описывает один блок blockMesh
    // q содержит четыре точки в плоскости, c - число ячеек по двум направлениям
    const blocks = [];
    const B = (q, c, tag) => blocks.push({ q, c, tag });
    // Большой шестиугольник разбиваем на центральные полосы и крайние части
    // Справа оставляется окно для соединительного канала
    B(['B_LL', 'B_LR', 'B_CH_B', 'B_LE_B'], [nXbig, nYbig], 'big_Rbot');
    B(['B_LE_B', 'B_CH_B', 'B_CH_T', 'B_LE_T'], [nXbig, nW], 'big_Rmid');
    B(['B_LE_T', 'B_CH_T', 'B_UR', 'B_UL'], [nXbig, nYbig], 'big_Rtop');
    // Нижняя часть большого шестиугольника схлопывается к крайней вершине
    B(['B_LR', 'B_LL', 'B_BOT', 'B_LR'], [nXbig, nCapB], 'big_capBot');
    // Верхняя часть разбита на три блока из-за входного патрубка
    B(['B_UL', 'B_UR', 'B_IB', 'B_UL'], [nXbig, nW], 'big_capW1');
    B(['B_UL', 'B_IB', 'B_IA', 'B_UL'], [nXbig, nW], 'big_capW2');
    B(['B_UL', 'B_IA', 'B_TOP', 'B_UL'], [nXbig, nW], 'big_capW3');
    // Малый шестиугольник строится аналогично
    // Слева находится окно соединительного канала
    B(['S_LL', 'S_LR', 'S_RE_B', 'S_CH_B'], [nXsm, nYsm], 'sm_Rbot');
    B(['S_CH_B', 'S_RE_B', 'S_RE_T', 'S_CH_T'], [nXsm, nW], 'sm_Rmid');
    B(['S_CH_T', 'S_RE_T', 'S_UR', 'S_UL'], [nXsm, nYsm], 'sm_Rtop');
    // Верхняя часть малого шестиугольника
    B(['S_UL', 'S_UR', 'S_TOP', 'S_UL'], [nXsm, nCapS], 'sm_capTop');
    // Нижняя часть разбита на три блока из-за выходного патрубка
    B(['S_LR', 'S_LL', 'S_OA', 'S_LR'], [nXsm, nW], 'sm_capU1');
    B(['S_LR', 'S_OA', 'S_OB', 'S_LR'], [nXsm, nW], 'sm_capU2');
    B(['S_LR', 'S_OB', 'S_BOT', 'S_LR'], [nXsm, nW], 'sm_capU3');
  }
}

```

```

// Отдельно добавляем прямой канал и два наклонных патрубка
B(['B_CH_B', 'S_CH_B', 'S_CH_T', 'B_CH_T'], [nChanL, nW], 'channel');
B(['B_IA', 'B_IB', 'B_FB', 'B_FA'], [nW, nPipeL], 'inletPipe');
B(['S_OB', 'S_OA', 'S_FA', 'S_FB'], [nW, nPipeL], 'outletPipe');
// Формируем строки hex для раздела blocks
// По Z всегда используется одна ячейка, так как задача двумерная
const blocksStr = blocks.map(b => {
  const [a, c, d, e] = b.q;
  return `hex (${vm(a)} ${vm(c)} ${vm(d)} ${vm(e)} ${vp(a)} ${vp(c)} ${vp(d)}
${vp(e)}) (${b.c[0]} ${b.c[1]} 1) simpleGrading (1 1 1) // ${b.tag}`;
}).join('\n');
// Преобразуем одно двумерное ребро в боковую грань трёхмерной сетки
const face = (a, b) => `${vm(a)} ${vm(b)} ${vp(b)} ${vp(a)}`;
// Ключ позволяет понять, сколько блоков используют одно и то же ребро
const ekey = (a, b) => [idx(a), idx(b)].sort((x, y) => x - y).join('-');
// Считаем количество повторений каждого бокового ребра
// Внутренние рёбра встречаются два раза, внешние только один
const sideCount = new Map();
const sideRepr = new Map();
for (const b of blocks) {
  const [p0, p1, p2, p3] = b.q;
  const edges = [[p0, p1], [p1, p2], [p2, p3], [p3, p0]];
  for (const [a, c] of edges) {
    // Схлопнутые рёбра не образуют настоящую граничную грань
    if (idx(a) === idx(c)) continue;
    const k = ekey(a, c);
    sideCount.set(k, (sideCount.get(k) || 0) + 1);
    if (!sideRepr.has(k)) sideRepr.set(k, [a, c]);
  }
}
// Отдельно задаём внешние грани входа и выхода
const inletPair = ['B_FB', 'B_FA'];
const outletPair = ['S_FA', 'S_FB'];
const inletK = ekey(...inletPair);
const outletK = ekey(...outletPair);
const inletFaces = [face(...inletPair)];
const outletFaces = [face(...outletPair)];
// Все остальные внешние боковые грани относятся к стенкам
const wallFaces = [];
for (const [k, cnt] of sideCount) {
  if (cnt !== 1) continue;
  if (k === inletK || k === outletK) continue;
  const [a, c] = sideRepr.get(k);
  wallFaces.push(face(a, c));
}
// Передняя и задняя стороны получают тип empty
// Благодаря этому OpenFOAM решает задачу только в плоскости XY
const fbFaces = [];
for (const b of blocks) {
  const [a, c, d, e] = b.q;
  fbFaces.push(`${vm(a)} ${vm(c)} ${vm(d)} ${vm(e)}`);
  fbFaces.push(`${vp(a)} ${vp(c)} ${vp(d)} ${vp(e)}`);
}

```

```

    }
    // Добавляем стандартный отступ перед строками граней
    const ind = (arr) => arr.map(s => `      ${s}`).join("\n");
    // Собираем окончательный blockMeshDict
    const out = `/*-----*- C++ -*-----*\\
=====
|
\\ \\   / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\ \\   / O peration   | Website: https://openfoam.org
\\ \\   / A nd         | Version: 9
\\ \\   M anipulation |
\\*-----*/
FoamFile
{
    format    ascii;
    class     dictionary;
    object     blockMeshDict;
}
convertToMeters 0.001;
vertices
(
    ${vg.pointsToString()}
);
blocks
(
    ${blocksStr}
);
edges
(
);
boundary
(
    inlet
    {
        type patch;
        faces
        (
            ${ind(inletFaces)}
        );
    }
    outlet
    {
        type patch;
        faces
        (
            ${ind(outletFaces)}
        );
    }
    walls
    {
        type wall;
        faces
        (

```

```

    ${ind(wallFaces)}
    );
  }
  frontAndBack
  {
    type empty;
    faces
    (
    ${ind(fbFaces)}
    );
  }
);
mergePatchPairs
(
);
// *****
//
`;
    // Сохраняем готовый словарь непосредственно в расчётный кейс
    const target = path.join(casePath, 'system', 'blockMeshDict');
    fs.writeFileSync(target, out, 'utf8');
    // Короткий вывод нужен для журнала веб-приложения
    console.log(`Сетка: ${blocks.length} блоков, ${vg.names.length * 2} вершин`);
  }
}
module.exports = { MeshGenerator };

```


Приложение В – Код файла «conditionsGenerator.js»

```
const fs = require('fs');
const path = require('path');
const SQRT3_2 = Math.sqrt(3) / 2;
function header(fieldClass, objectName) {
    return `/*-----*- C++ -*-----*\\
=====
\\\\\\ / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\\\\\ / O peration   | Website: https://openfoam.org
\\\\\\ / A nd          | Version: 9
\\\\\\ M anipulation  |
\\*-----*/

FoamFile
{
    format    ascii;
    class     ${fieldClass};
    location  "0";
    object     ${objectName};
}
// *****
`;
}

class ConditionsGenerator {
    exportU(speed, casePath) {
        // Входной патрубок расположен под углом 60 градусов.
        const ux = (-0.5 * speed).toFixed(6);
        const uy = (-SQRT3_2 * speed).toFixed(6);

        const text = `${header('volVectorField', 'U')}dimensions    [0 1 -1 0 0 0];
internalField    uniform (0 0 0);
boundaryField
{
    inlet
    {
        type        fixedValue;
        value        uniform (${ux} ${uy} 0);
    }
    outlet
    {
        type        zeroGradient;
    }
    walls
    {
        type        noSlip;
    }
    frontAndBack
    {
        type        empty;
    }
}
// *****
`;
```

```

    fs.writeFileSync(path.join(casePath, '0', 'U'), text, 'utf8');
    console.log('Файл U создан');
  }
  exportP(casePath) {
    const text = `${header('volScalarField', 'p')}dimensions    [0 2 -2 0 0 0 0];
internalField    uniform 0;
boundaryField
{
  inlet
  {
    type          zeroGradient;
  }
  outlet
  {
    type          fixedValue;
    value         uniform 0;
  }
  walls
  {
    type          zeroGradient;
  }
  frontAndBack
  {
    type          empty;
  }
}

// *****
//
`;
    fs.writeFileSync(path.join(casePath, '0', 'p'), text, 'utf8');
    console.log('Файл p создан');
  }
  exportAll(speed, casePath) {
    this.exportU(speed, casePath);
    this.exportP(casePath);
  }
}
module.exports = { ConditionsGenerator };

```

```

const express = require('express');
const { spawn } = require('child_process');
const path = require('path');
const app = express();
const PORT = 3000;
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));
let currentStatus = 'idle';
let currentLog = "";
let currentStage = 'Ожидание';
let child = null;
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});
app.post('/calculate', (req, res) => {
  if (currentStatus === 'running') {
    return res.status(409).json({ success: false, error: 'Расчёт уже выполняется' });
  }
  const { sBig, sSmall, lRect, lPipe, w, speed } = req.body;
  const values = [sBig, sSmall, lRect, lPipe, w, speed];
  if (values.some(value => value === undefined || value === null || value === "")) {
    return res.status(400).json({ success: false, error: 'Заполнены не все поля' });
  }
  currentStatus = 'running';
  currentLog = "";
  currentStage = 'Проверка параметров';
  res.json({ success: true, message: 'Расчёт запущен' });
  const args = values.map(String);
  child = spawn('node', [path.join(__dirname, 'run.js'), ...args], {
    cwd: __dirname
  });
  const appendOutput = data => {
    const text = data.toString();
    currentLog += text;
    const stages = [
      'Проверка параметров',
      'Создание геометрии',
      'Построение сетки',
      'Проверка сетки',
      'Расчёт simpleFoam',
      'Создание изображений',
      'Готово'
    ];
    for (const stage of stages) {
      if (text.includes(stage)) currentStage = stage;
    }
  };
  child.stdout.on('data', appendOutput);
  child.stderr.on('data', appendOutput);
  child.on('error', err => {

```

```

    currentLog += '\nОшибка запуска: ${err.message}\n';
    currentStatus = 'error';
    currentStage = 'Ошибка запуска';
    child = null;
  });
  child.on('close', code => {
    currentStatus = code === 0 ? 'done' : 'error';
    currentStage = code === 0 ? 'Готово' : 'Ошибка расчёта';
    child = null;
  });
});
app.get('/status', (req, res) => {
  res.json({
    status: currentStatus,
    stage: currentStage,
    log: currentLog
  });
});
app.listen(PORT, () => {
  console.log(`Сервер запущен: http://localhost:${PORT}`);
});

```

```
#!/usr/bin/env node
const fs = require('fs');
const path = require('path');
const { spawn } = require('child_process');
const { VerticesGenerator } = require('./formGenerator');
const { MeshGenerator } = require('./meshGenerator');
const { ConditionsGenerator } = require('./conditionsGenerator');
const ROOT = __dirname;
const CASE = path.join(ROOT, 'case');
const PUBLIC = path.join(ROOT, 'public');
const SQRT3_2 = Math.sqrt(3) / 2;
function fail(message) {
  console.error(`Ошибка: ${message}`);
  process.exit(1);
}
function runCommand(command, args, cwd, extraEnv = {}) {
  return new Promise((resolve, reject) => {
    const child = spawn(command, args, {
      cwd,
      stdio: ['ignore', 'pipe', 'pipe'],
      env: { ...process.env, ...extraEnv }
    });
    child.stdout.on('data', data => process.stdout.write(data));
    child.stderr.on('data', data => process.stderr.write(data));
    child.on('error', reject);
    child.on('close', code => {
      if (code === 0) resolve();
      else reject(new Error(`${command} завершился с кодом ${code}`));
    });
  });
}
function removeOldImages() {
  for (const name of ['velocity.png', 'velocity_x.png', 'pressure.png', 'streamlines.png']) {
    const file = path.join(PUBLIC, name);
    if (fs.existsSync(file)) fs.unlinkSync(file);
  }
}
async function makeImages() {
  const script = path.join(ROOT, 'visualize.py');
  const args = [script, CASE, PUBLIC];
  try {
    await runCommand('pvbatch', args, ROOT);
    return true;
  } catch (_) {
    try {
      await runCommand('pvbatch', args, ROOT, { LIBGL_DRI3_DISABLE: 'true' });
      return true;
    } catch (_) {
      return false;
    }
  }
}
```

```

    }
  }
  (async () => {
    console.log('Проверка параметров');
    const values = process.argv.slice(2).map(Number);
    if (values.length !== 6 || values.some(value => !Number.isFinite(value))) {
      fail('неверно заданы параметры');
    }
    const [sBig, sSmall, lRect, lPipe, w, speed] = values;
    if (values.some(value => value <= 0)) fail('все значения должны быть больше нуля');
    if (sSmall >= sBig) fail('малый шестиугольник должен быть меньше большого');
    if (w >= sSmall) fail('ширина канала слишком большая');
    const aBig = sBig * SQRT3_2;
    const aSmall = sSmall * SQRT3_2;
    const channelLen = lRect - aBig - aSmall;
    if (channelLen <= 0) fail('L_rect слишком мало для заданных S_big и S_small');
    const nu = 1.5e-5;
    const reynolds = speed * (w / 1000) / nu;
    console.log(`Размеры: ${sBig}, ${sSmall}, ${lRect}, ${lPipe}, ${w} мм`);
    console.log(`Скорость: ${speed} м/с`);
    console.log(`Re = ${reynolds.toFixed(1)}`);
    console.log('Создание геометрии');
    try {
      const geometry = new VerticesGenerator(sBig, sSmall, lRect, lPipe,
w).generateVertices();
      new MeshGenerator().exportBlockMeshDict(geometry, CASE);
      new ConditionsGenerator().exportAll(speed, CASE);
    } catch (error) {
      fail(error.message);
    }
    removeOldImages();
    try {
      await runCommand('bash', [path.join(ROOT, 'Run.sh')], ROOT);
    } catch (error) {
      fail(error.message);
    }
    console.log('Создание изображений');
    const imagesReady = await makeImages();
    if (!imagesReady) console.log('Изображения не созданы');
    console.log('Готово');
  })().catch(error => fail(error && error.message ? error.message : String(error)));

```

Приложение Е – Код файла «validation.js»

```
// Кинематическая вязкость воздуха при комнатной температуре, м2/с
const AIR_NU = 1.5e-5;
// Верхняя граница ламинарного режима для этого проекта
const MAX_RE = 2300;
// Постоянная используется при расчёте апофемы шестиугольника
const Sqrt3_2 = Math.sqrt(3) / 2;
// Идентификаторы всех редактируемых полей
const ids = ['sBig', 'sSmall', 'lRect', 'lPipe', 'w', 'speed'];
// Не даём запустить второй расчёт, пока работает первый
let calculationRunning = false;
// После загрузки страницы задаём исходные параметры и подключаем обработчики
document.addEventListener('DOMContentLoaded', () => {
  // Исходные параметры варианта
  const defaults = { sBig: 20, sSmall: 10, lRect: 35, lPipe: 15, w: 4, speed: 3 };
  // Записываем исходные значения в поля
  Object.entries(defaults).forEach(([id, value]) => {
    document.getElementById(id).value = value;
  });
  // Основная кнопка запуска
  document.getElementById('calculate-btn').addEventListener('click', validateAndRun);
  // При изменении параметров сразу пересчитываем Re и проверяем геометрию
  ids.forEach(id => {
    document.getElementById(id).addEventListener('input', updateInfo);
  });
  // Стрелки справа работают одинаково для всех числовых полей
  document.querySelectorAll('.number-step').forEach(button => {
    button.addEventListener('click', () => {
      const input = document.getElementById(button.dataset.target);
      const direction = Number(button.dataset.direction);
      if (direction > 0) input.stepUp();
      else input.stepDown();
      input.dispatchEvent(new Event('input', { bubbles: true }));
    });
  });
  // Заполняем информационные блоки сразу после открытия страницы
  updateInfo();
});
// Читаем введённые пользователем параметры
function readValues() {
  return {
    sBig: Number(document.getElementById('sBig').value),
    sSmall: Number(document.getElementById('sSmall').value),
    lRect: Number(document.getElementById('lRect').value),
    lPipe: Number(document.getElementById('lPipe').value),
    w: Number(document.getElementById('w').value),
    speed: Number(document.getElementById('speed').value)
  };
}
// Число Рейнольдса считается по ширине канала
function reynolds(p) {
```

```

// W задаётся в миллиметрах, поэтому переводим его в метры
return p.speed * (p.w / 1000) / AIR_NU;
}
// Проверяем, можно ли построить геометрию с введёнными размерами
function checkGeometry(p) {
    // Все размеры должны быть положительными числами
    if ([p.sBig, p.sSmall, p.lRect, p.lPipe, p.w].some(v => !Number.isFinite(v) || v <= 0)) {
        return { ok: false, text: 'Все размеры должны быть больше нуля' };
    }
    // Малый шестиугольник должен быть меньше большого
    if (p.sSmall >= p.sBig) {
        return { ok: false, text: 'S_small должен быть меньше S_big' };
    }
    // Ширина канала должна быть меньше малого шестиугольника
    if (p.w >= p.sSmall) {
        return { ok: false, text: 'W должен быть меньше S_small' };
    }
    // Апофемы двух правильных шестиугольников
    const aBig = p.sBig * SQRT3_2;
    const aSmall = p.sSmall * SQRT3_2;
    // Длина прямого участка между двумя камерами
    const length = p.lRect - aBig - aSmall;
    // Если значение отрицательное, камеры пересекаются
    if (length <= 0) {
        return { ok: false, text: `Увеличьте L_rect минимум до ${ (aBig + aSmall).toFixed(1)} мм` };
    }
    // Слишком короткий участок ухудшает качество сетки
    if (length < p.w) {
        return { ok: false, text: `Увеличьте L_rect минимум до ${ (aBig + aSmall + p.w).toFixed(1)} мм` };
    }
    // Геометрия подходит для дальнейшего расчёта
    return { ok: true, length };
}
// Обновляем Re, проверку геометрии и состояние кнопки
function updateInfo() {
    // Берём текущие значения и считаем основные проверки
    const p = readValues();
    const geometry = checkGeometry(p);
    const re = reynolds(p);
    // Элементы интерфейса
    const reBox = document.getElementById('reynolds-box');
    const geometryBox = document.getElementById('geometry-box');
    const geometryCard = document.getElementById('geometry-card');
    const button = document.getElementById('calculate-btn');
    // Показываем режим течения
    if (Number.isFinite(re) && re > 0) {
        reBox.textContent = re < MAX_RE
            ? `Re ≈ ${re.toFixed(0)} – ламинарный режим`
            : `Re ≈ ${re.toFixed(0)} – вне ламинарного диапазона`;
    } else {

```



```

    reBox.textContent = 'Введите W и U';
  }
  // Показываем результат проверки геометрии
  if (geometry.ok) {
    geometryBox.innerHTML = `<span class="geometry-main">Геометрия
корректна</span><span class="geometry-detail">Прямой участок:
${geometry.length.toFixed(1)} мм</span>`;
    geometryCard.classList.remove('invalid');
  } else {
    geometryBox.textContent = geometry.text;
    geometryCard.classList.add('invalid');
  }
  // Кнопка доступна только для корректной геометрии и Re меньше 2300
  button.disabled = calculationRunning || !geometry.ok || !Number.isFinite(re) || re >=
MAX_RE;
}
// Убираем старые ошибки перед новой проверкой
function clearErrors() {
  ids.forEach(id => {
    document.getElementById(`${id}-error`).textContent = "";
  });
}
// Показываем ошибку возле нужного поля
function setError(id, text) {
  document.getElementById(`${id}-error`).textContent = text;
}
// Проверяем данные непосредственно перед запуском расчёта
function validateAndRun() {
  // Удаляем старые сообщения и ещё раз читаем параметры
  clearErrors();
  const p = readValues();
  const geometry = checkGeometry(p);
  const re = reynolds(p);
  let ok = true;
  // Проверяем каждое числовое поле
  ids.forEach(id => {
    if (!Number.isFinite(p[id]) || p[id] <= 0) {
      setError(id, 'Введите значение больше нуля');
      ok = false;
    }
  });
  // Проверяем соотношение размеров камер
  if (p.sSmall >= p.sBig) {
    setError('sSmall', 'S_small должен быть меньше S_big');
    ok = false;
  }
  // Проверяем ширину канала
  if (p.w >= p.sSmall) {
    setError('w', 'W должен быть меньше S_small');
    ok = false;
  }
  // Ошибку расстояния показываем под L_rect

```

```

if (!geometry.ok && geometry.text.includes('L_rect')) {
    setError('lRect', geometry.text);
    ok = false;
}
// Ламинарный расчёт не запускаем при Re выше границы
if (Number.isFinite(re) && re >= MAX_RE) {
    setError('speed', 'Для расчёта требуется Re < 2300');
    ok = false;
}
// Если всё правильно, запускаем расчёт
if (ok) startCalculation(p);
else updateInfo();
}
// Отправляем параметры на Node.js сервер
async function startCalculation(params) {
    // Получаем элементы, которые меняются во время расчёта
    const button = document.getElementById('calculate-btn');
    const status = document.getElementById('status-box');
    const log = document.getElementById('log-box');
    const results = document.getElementById('result-images');
    // Блокируем повторный запуск и прячем старые результаты
    calculationRunning = true;
    button.disabled = true;
    status.textContent = 'Расчёт';
    log.textContent = '';
    results.style.display = 'none';
    try {
        // Передаём параметры серверу в формате JSON
        const response = await fetch('/calculate', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(params)
        });
        const data = await response.json();
        // Если сервер не смог запустить процесс, показываем ошибку
        if (!response.ok || !data.success) {
            throw new Error(data.error || 'Не удалось запустить расчёт');
        }
        // После запуска начинаем следить за состоянием
        pollStatus();
    } catch (error) {
        calculationRunning = false;
        status.textContent = 'Ошибка расчёта';
        log.textContent = error.message;
        updateInfo();
    }
}
// Получаем состояние расчёта от сервера
function pollStatus() {
    const status = document.getElementById('status-box');
    const log = document.getElementById('log-box');
    const results = document.getElementById('result-images');

```

```

// Проверяем состояние примерно раз в 1.2 секунды
const timer = setInterval(async () => {
  try {
    const response = await fetch('/status');
    const data = await response.json();
    // Обновляем короткий статус и журнал
    status.textContent = data.stage || data.status;
    log.textContent = data.log || "";
    log.scrollTop = log.scrollHeight;
    // Расчёт завершился успешно
    if (data.status === 'done') {
      clearInterval(timer);
      calculationRunning = false;
      status.textContent = 'Готово';
      // Время в адресе не даёт браузеру показать старые PNG из кеша
      const t = Date.now();
      setImage('img-velocity', `velocity.png?t=${t}`);
      setImage('img-velocity-x', `velocity_x.png?t=${t}`);
      setImage('img-pressure', `pressure.png?t=${t}`);
      setImage('img-streamlines', `streamlines.png?t=${t}`);
      // Показываем результаты и возвращаем правильное состояние кнопки
      results.style.display = 'block';
      updateInfo();
      results.scrollToView({ behavior: 'smooth' });
    }
    // Сервер сообщил об ошибке расчёта
    if (data.status === 'error') {
      clearInterval(timer);
      calculationRunning = false;
      status.textContent = 'Ошибка расчёта';
      updateInfo();
    }
  } catch {
    // Ошибка связи с Node.js сервером
    clearInterval(timer);
    calculationRunning = false;
    status.textContent = 'Ошибка соединения';
    updateInfo();
  }
}, 1200);
}

// Устанавливаем изображение результата
function setImage(id, src) {
  const img = document.getElementById(id);
  img.src = src;
  // Если PNG не создан, вместо пустой картинки остаётся поясняющий текст
  img.onerror = () => {
    img.removeAttribute('src');
    img.alt = 'Изображение не создано';
  };
}

```

```
#!/usr/bin/env pvbatch
# -*- coding: utf-8 -*-
# Автоматическое построение изображений результатов OpenFOAM
import os
import sys
# Получаем пути к расчётному кейсу и папке с изображениями
case_dir = sys.argv[1] if len(sys.argv) > 1 else "./case"
out_dir = sys.argv[2] if len(sys.argv) > 2 else "./public"
foam_file = os.path.join(case_dir, "foam.foam")
# Файл foam.foam используется ParaView для открытия результатов OpenFOAM
if not os.path.exists(foam_file):
    open(foam_file, "a").close()
# Создаём каталог для изображений, если его ещё нет
if not os.path.isdir(out_dir):
    os.makedirs(out_dir)
# Подключаем ParaView
import paraview.simple as pvs
from paraview.simple import *
# Отключаем автоматическое изменение камеры после первого отображения
pvs._DisableFirstRenderCameraReset()
# Загружаем внутреннюю расчётную область OpenFOAM
reader = OpenFOAMReader(FileName=foam_file)
try:
    reader.MeshRegions = ['internalMesh']
except Exception:
    pass
# Для визуализации используются поля скорости U и давления p
try:
    reader.CellArrays = ['U', 'p']
except Exception:
    pass
reader.UpdatePipeline()
# Определяем последний сохранённый шаг расчёта
scene = GetAnimationScene()
scene.UpdateAnimationUsingDataTimeSteps()
time_keeper = GetTimeKeeper()
times = list(time_keeper.TimestepValues) if time_keeper.TimestepValues is not None else []
last_time = times[-1] if times else 0.0
scene.AnimationTime = last_time
# Создаём окно визуализации
view = GetActiveViewOrCreate('RenderView')
view.ViewSize = [1280, 720]
view.Background = [1.0, 1.0, 1.0]
view.OrientationAxesVisibility = 0
view.CameraParallelProjection = 1
try:
    view.InteractionMode = '2D'
except Exception:
    pass
view.ViewTime = last_time
```

```

# Обновляем результаты последнего шага
reader.UpdatePipeline(last_time)
# Переводим значения из центров ячеек в узлы
# Это делает цветовые переходы более плавными
cell2point = CellDatatoPointData(Input=reader)
cell2point.UpdatePipeline(last_time)
# Получаем размеры расчётной области
bounds = cell2point.GetDataInformation().GetBounds()
xmin, xmax, ymin, ymax, zmin, zmax = bounds
xr = xmax - xmin
yr = ymax - ymin
cx = 0.5 * (xmin + xmax)
cy = 0.5 * (ymin + ymax)
zmid = 0.5 * (zmin + zmax)
# Устанавливаем одинаковый вид сверху для всех изображений
def set_camera():
    aspect = 1280.0 / 720.0
    scale_y = 0.5 * yr * 1.13
    scale_x = 0.5 * xr / aspect * 1.13
    scale = max(scale_y, scale_x, 1e-6)
    # Небольшое смещение вверх оставляет место для шкалы снизу
    camera_y = cy - 0.055 * yr
    distance = max(xr, yr, 1e-3) * 10.0
    view.CameraFocalPoint = [cx, camera_y, zmid]
    view.CameraPosition = [cx, camera_y, zmid + distance]
    view.CameraViewUp = [0.0, 1.0, 0.0]
    view.CameraParallelProjection = 1
    view.CameraParallelScale = scale
    Render()
# Применяем одинаковую радужную палитру
def apply_rainbow(field):
    lut = GetColorTransferFunction(field)
    try:
        lut.ApplyPreset('Blue to Red Rainbow', True)
    except Exception:
        try:
            lut.ApplyPreset('Rainbow Uniform', True)
        except Exception:
            pass
    return lut
# Настраиваем горизонтальную цветовую шкалу
def style_bar(lut, title):
    try:
        bar = GetScalarBar(lut, view)
    except Exception:
        return
    # Название отображаемой величины
    try:
        bar.Title = title
    except Exception:
        pass
    # Дополнительная подпись компоненты не нужна

```

```

try:
    bar.ComponentTitle = "
except Exception:
    pass
# Чёрный текст хорошо читается на белом фоне
try:
    bar.TitleColor = [0.0, 0.0, 0.0]
except Exception:
    pass
try:
    bar.LabelColor = [0.0, 0.0, 0.0]
except Exception:
    pass
# Делаем шкалу горизонтальной
try:
    bar.Orientation = 'Horizontal'
except Exception:
    pass
# Для старых версий ParaView используется AnyLocation без пробела
try:
    bar.WindowLocation = 'AnyLocation'
except Exception:
    try:
        bar.WindowLocation = 'Any Location'
    except Exception:
        pass
# Размер шкалы
try:
    bar.ScalarBarLength = 0.42
except Exception:
    pass
try:
    bar.ScalarBarThickness = 16
except Exception:
    pass
# Размер текста
try:
    bar.TitleFontSize = 14
except Exception:
    pass
try:
    bar.LabelFontSize = 11
except Exception:
    pass
# Шкала длиной 0.42 центрируется по горизонтали при X = 0.29
# Y задаёт небольшой отступ от нижней границы изображения
try:
    bar.Position = [0.29, 0.045]
except Exception:
    pass
# Прячем предыдущие объекты перед построением следующего изображения
def clear_view():

```

```

try:
    Hide(reader, view)
except Exception:
    pass
try:
    Hide(cell2point, view)
except Exception:
    pass
try:
    HideUnusedScalarBars(view)
except Exception:
    pass
# Создаём изображение поля скорости или давления
def save_field(field, component, filename, title):
    try:
        clear_view()
        # Используем значения, интерполированные в узлы сетки
        # Это убирает ступенчатость отдельных ячеек
        display = Show(cell2point, view)
        display.Representation = 'Surface'
        # Для скорости можно вывести модуль либо компоненту X
        if component:
            ColorBy(display, ('POINTS', field, component))
        else:
            ColorBy(display, ('POINTS', field))
        # Диапазон шкалы определяется по текущему расчёту
        display.RescaleTransferFunctionToDataRange(True, False)
        display.SetScalarBarVisibility(view, True)
        lut = apply_rainbow(field)
        style_bar(lut, title)
        set_camera()
        # Сохраняем готовое изображение
        SaveScreenshot(
            os.path.join(out_dir, filename),
            view,
            ImageResolution=[1280, 720]
        )
        print("Сохранено: %s" % filename)
    except Exception:
        pass
    Hide(cell2point, view)
except Exception as error:
    print("Не удалось создать %s: %s" % (filename, error))
# Формируем основные изображения результатов
save_field('U', 'Magnitude', 'velocity.png', '|U| (m/s)')
save_field('U', 'X', 'velocity_x.png', 'Ux (m/s)')
save_field('p', None, 'pressure.png', 'p (m^2/s^2)')
# Строим линии тока в средней плоскости двумерной модели
def save_streamlines():
    try:

```

```

clear_view()
# Форму расчётной области оставляем светлым фоном
background = Show(cell2point, view)
background.Representation = 'Surface'
background.Opacity = 0.20
background.AmbientColor = [0.68, 0.68, 0.68]
background.DiffuseColor = [0.68, 0.68, 0.68]
try:
    ColorBy(background, None)
except Exception:
    pass
# Модель имеет одну ячейку по Z, поэтому стартовые точки размещаем строго в
средней плоскости
# Пять линий-затравок распределяются по всей расчётной области
seed_positions = [0.18, 0.34, 0.50, 0.66, 0.82]
displays = []
lut = apply_rainbow('U')
# Для каждой затравочной линии создаём свой StreamTracer
for part in seed_positions:
    x = xmin + part * xr
    seed = pvs.Line()
    seed.Point1 = [x, ymin + 0.06 * yr, zmid]
    seed.Point2 = [x, ymax - 0.06 * yr, zmid]
    # 15 начальных точек дают подробную картину без лишнего перегруза
    seed.Resolution = 14
    tracer = pvs.StreamTracerWithCustomSource(
        Input=cell2point,
        SeedSource=seed
    )
    tracer.Vectors = ['POINTS', 'U']
    # Интегрирование в обе стороны показывает прямой и возвратный поток
    tracer.IntegrationDirection = 'BOTH'
    tracer.MaximumStreamlineLength = max(xr, yr) * 6.0
    try:
        tracer.MaximumSteps = 5000
    except Exception:
        pass
    tracer.UpdatePipeline(last_time)
    # Окрашиваем линии тока по горизонтальной составляющей скорости
    display = Show(tracer, view)
    display.Representation = 'Surface'
    ColorBy(display, ('POINTS', 'U', 'X'))
    display.RescaleTransferFunctionToDataRange(True, False)
    try:
        display.LineWidth = 1.0
    except Exception:
        pass
    try:
        display.RenderLinesAsTubes = 0
    except Exception:
        pass
    displays.append(display)

```



```

# Для всех линий используется одна общая цветовая шкала
if displays:
    displays[0].SetScalarBarVisibility(view, True)
    style_bar(lut, 'Ux (m/s)')
# Добавляем контур расчётной области, чтобы на изображении были видны стенки
# Сначала извлекаем наружную поверхность модели
surface = ExtractSurface(Input=reader)
surface.UpdatePipeline(last_time)
# FeatureEdges оставляет характерные рёбра наружной поверхности без
отображения сетки
walls = FeatureEdges(Input=surface)
try:
    walls.BoundaryEdges = 1
except Exception:
    pass
try:
    walls.FeatureEdges = 1
except Exception:
    pass
try:
    walls.NonManifoldEdges = 0
except Exception:
    pass
try:
    walls.ManifoldEdges = 0
except Exception:
    pass
walls.UpdatePipeline(last_time)
walls_display = Show(walls, view)
try:
    ColorBy(walls_display, None)
except Exception:
    pass
try:
    walls_display.LineWidth = 2.0
except Exception:
    pass
try:
    walls_display.AmbientColor = [0.22, 0.22, 0.22]
except Exception:
    pass
try:
    walls_display.DiffuseColor = [0.22, 0.22, 0.22]
except Exception:
    pass
try:
    walls_display.SetScalarBarVisibility(view, False)
except Exception:
    pass
set_camera()
# Сохраняем изображение линий тока вместе с контуром стенок
SaveScreenshot(

```

```
        os.path.join(out_dir, 'streamlines.png'),
        view,
        ImageResolution=[1280, 720]
    )
    print("Сохранено: streamlines.png")
except Exception as error:
    print("Не удалось создать streamlines.png: %s" % error)
# Линии тока формируются после основных полей
save_streamlines()
print("Изображения готовы")
```

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Расчёт течения воздуха через канал</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
<main class="page">
  <!-- Заголовок приложения -->
  <h1>Расчёт течения воздуха через канал</h1>
  <!-- Верхняя часть: схема слева, параметры справа -->
  <section class="top-grid">
    <!-- Расчётная схема -->
    <section class="card scheme-card">
      <div class="card-head">
        <h2>Расчётная схема</h2>
        <p>Параметрическая геометрия расчётной области.</p>
      </div>
      
    </section>
    <!-- Параметры расчёта -->
    <section class="card controls">
      <div class="card-head">
        <h2>Параметры расчёта</h2>
        <p>Геометрические параметры и скорость воздуха.</p>
      </div>
      <!-- Поля исходных данных -->
      <div class="inputs">
        <!-- Сторона большого шестиугольника -->
        <label>
          <span>S_big – сторона большого шестиугольника</span>
          <div class="input-unit">
            <input type="number" id="sBig" value="20" min="0.1" step="0.1">
            <span class="unit">мм</span>
            <div class="number-controls">
              <button type="button" class="number-step" data-target="sBig" data-direction="1" aria-label="Увеличить S_big">▲</button>
              <button type="button" class="number-step" data-target="sBig" data-direction="-1" aria-label="Уменьшить S_big">▼</button>
            </div>
          </div>
          <small id="sBig-error"></small>
        </label>
        <!-- Сторона малого шестиугольника -->
        <label>
          <span>S_small – сторона малого шестиугольника</span>
          <div class="input-unit">
            <input type="number" id="sSmall" value="10" min="0.1" step="0.1">

```

```

        <span class="unit">мм</span>
        <div class="number-controls">
            <button type="button" class="number-step" data-target="sSmall" data-
direction="1" aria-label="Увеличить S_small">▲</button>
            <button type="button" class="number-step" data-target="sSmall" data-
direction="-1" aria-label="Уменьшить S_small">▼</button>
        </div>
    </div>
    <small id="sSmall-error"></small>
</label>
<!-- Расстояние между центрами -->
<label>
    <span>L_rect – расстояние между центрами</span>
    <div class="input-unit">
        <input type="number" id="lRect" value="35" min="0.1" step="0.1">
        <span class="unit">мм</span>
        <div class="number-controls">
            <button type="button" class="number-step" data-target="lRect" data-
direction="1" aria-label="Увеличить L_rect">▲</button>
            <button type="button" class="number-step" data-target="lRect" data-
direction="-1" aria-label="Уменьшить L_rect">▼</button>
        </div>
    </div>
    <small id="lRect-error"></small>
</label>
<!-- Длина патрубков -->
<label>
    <span>L_pipe – длина патрубков</span>
    <div class="input-unit">
        <input type="number" id="lPipe" value="15" min="0.1" step="0.1">
        <span class="unit">мм</span>
        <div class="number-controls">
            <button type="button" class="number-step" data-target="lPipe" data-
direction="1" aria-label="Увеличить L_pipe">▲</button>
            <button type="button" class="number-step" data-target="lPipe" data-
direction="-1" aria-label="Уменьшить L_pipe">▼</button>
        </div>
    </div>
    <small id="lPipe-error"></small>
</label>
<!-- Ширина канала -->
<label>
    <span>W – ширина канала</span>
    <div class="input-unit">
        <input type="number" id="w" value="4" min="0.1" step="0.1">
        <span class="unit">мм</span>
        <div class="number-controls">
            <button type="button" class="number-step" data-target="w" data-
direction="1" aria-label="Увеличить W">▲</button>
            <button type="button" class="number-step" data-target="w" data-
direction="-1" aria-label="Уменьшить W">▼</button>
        </div>
    </div>

```

```

        </div>
        <small id="w-error"></small>
    </label>
    <!-- Скорость воздуха -->
    <label>
        <span>U – скорость воздуха на входе</span>
        <div class="input-unit">
            <input type="number" id="speed" value="3" min="0.01" step="0.1">
            <span class="unit">м/с</span>
            <div class="number-controls">
                <button type="button" class="number-step" data-target="speed" data-
direction="1" aria-label="Увеличить U">▲</button>
                <button type="button" class="number-step" data-target="speed" data-
direction="-1" aria-label="Уменьшить U">▼</button>
            </div>
        </div>
        <small id="speed-error"></small>
    </label>
</div>
<!-- Информация о режиме и геометрии -->
<div class="summary">
    <div class="re-box" id="reynolds-box"></div>
    <div class="geometry-box" id="geometry-card">
        <strong>Проверка геометрии</strong>
        <div id="geometry-box"></div>
    </div>
</div>
<!-- Запуск расчёта -->
<button id="calculate-btn">Запустить расчёт</button>
</section>
</section>
<!-- Состояние расчёта -->
<section class="card process">
    <div class="process-head">
        <div class="process-info">
            <h2>Ход расчёта</h2>
            <p>Выполнение расчёта OpenFOAM.</p>
        </div>
        <div class="status" id="status-box">Ожидание</div>
    </div>
    <!-- Журнал скрыт до нажатия -->
    <details class="log-details">
        <summary>Показать журнал</summary>
        <pre id="log-box"></pre>
    </details>
</section>
<!-- Результаты появляются после завершения расчёта -->
<section class="results" id="result-images">
    <div class="card-head">
        <h2>Результаты расчёта</h2>
    </div>
    <div class="results-grid">

```

```

    <figure>
      <figcaption>Модуль скорости  $|U|$ </figcaption>
      <img id="img-velocity" alt="Поле модуля скорости">
    </figure>
    <figure>
      <figcaption>Составляющая скорости  $U_x$ </figcaption>
      <img id="img-velocity-x" alt="Поле скорости  $U_x$ ">
    </figure>
    <figure>
      <figcaption>Кинематическое давление  $p$ </figcaption>
      <img id="img-pressure" alt="Поле давления">
    </figure>
    <figure>
      <figcaption>Линии тока</figcaption>
      <img id="img-streamlines" alt="Линии тока">
    </figure>
  </div>
</section>
</main>
<!-- Проверка параметров и обмен с сервером -->
<script src="validation.js"></script>
</body>
</html>

```

```

/* Размер элемента включает padding и border */
* {
  box-sizing: border-box;
}
/* Основные цвета интерфейса */
:root {
  --bg: #f1f4f3;
  --card: #ffffff;
  --line: #d4d4db;
  --text: #182321;
  --muted: #687774;
  --accent: #287e76;
  --accent-dark: #206860;
  --accent-light: #e7f2f0;
  --error: #b42318;
}
/* Общий вид страницы */
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: var(--bg);
  color: var(--text);
}
/* Основной контейнер */
.page {
  width: min(1360px, calc(100% - 40px));
  margin: 0 auto;
  padding: 24px 0 40px;
}
/* Главный заголовок */
h1 {
  margin: 0 0 18px;
  font-size: 30px;
}
/* Общий вид карточек */
.card,
.results {
  background: var(--card);
  border: 1px solid var(--line);
  border-radius: 14px;
  padding: 20px;
}
/* Схема и параметры располагаются рядом */
.top-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
  align-items: stretch;
}
/* Заголовок карточки */

```

```

.card-head {
    margin-bottom: 14px;
}
/* Заголовки разделов */
.card-head h2 {
    margin: 0 0 5px;
    font-size: 21px;
}
/* Обычный поясняющий текст */
.card-head p {
    margin: 0;
    color: var(--muted);
    font-size: 14px;
    font-weight: 400;
}
/* Расчётная схема */
.scheme {
    width: 100%;
    height: 430px;
    object-fit: contain;
    border: 1px solid #e3e9e7;
    border-radius: 10px;
    padding: 8px;
}
/* Карточка параметров растягивается по высоте */
.controls {
    display: flex;
    flex-direction: column;
    height: 100%;
}
/* Поля ввода располагаются в две колонки */
.inputs {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 14px 16px;
}
/* Подпись и поле одного параметра */
label {
    display: flex;
    flex-direction: column;
    gap: 6px;
    font-size: 14px;
}
/* Обёртка числового поля */
.input-unit {
    position: relative;
}
/* Само поле ввода */
.input-unit input {
    width: 100%;
    padding: 11px 78px 11px 12px;
    border: 1px solid #b8c5c2;
}

```



```

border-radius: 8px;
background: #ffffff;
color: var(--text);
font-size: 15px;
}
/* Убираем стандартные стрелки браузера */
.input-unit input[type="number"] {
  appearance: textfield;
  -moz-appearance: textfield;
}
.input-unit input[type="number"]::-webkit-inner-spin-button,
.input-unit input[type="number"]::-webkit-outer-spin-button {
  -webkit-appearance: none;
  margin: 0;
}
/* Подсветка активного поля */
.input-unit input:focus {
  outline: 2px solid #cfe4e1;
  border-color: var(--accent);
}
/* Единица измерения стоит перед стрелками */
.input-unit .unit {
  position: absolute;
  right: 38px;
  top: 50%;
  transform: translateY(-50%);
  color: #8a9794;
  font-size: 12px;
  pointer-events: none;
}
/* Стрелки находятся у правой стенки поля */
.number-controls {
  position: absolute;
  top: 1px;
  right: 1px;
  bottom: 1px;
  width: 30px;
  display: grid;
  grid-template-rows: 1fr 1fr;
  border-left: 1px solid #d6dfdd;
  border-radius: 0 7px 7px 0;
  overflow: hidden;
}
/* Верхняя и нижняя стрелки */
.number-controls .number-step {
  width: 100%;
  min-height: 0;
  margin: 0;
  padding: 0;
  border: 0;
  border-radius: 0;
  background: #f3f6f5;

```

```

    color: #7c8e8a;
    font-size: 8px;
    line-height: 1;
    cursor: pointer;
}
/* Разделитель между стрелками */
.number-controls .number-step:first-child {
    border-bottom: 1px solid #d6dfdd;
}
/* Подсветка стрелки */
.number-controls .number-step:hover {
    background: #e7efed;
    color: var(--accent-dark);
}
/* Ошибка под полем */
small {
    color: var(--error);
    font-size: 12px;
}
/* Пустая ошибка не занимает место */
small:empty {
    display: none;
}
/* Блок Re и проверки геометрии */
.summary {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 14px;
    margin-top: 16px;
}
/* Общий вид информационных карточек */
.re-box,
.geometry-box {
    min-height: 74px;
    padding: 12px 14px;
    border: 1px solid #dce5e3;
    border-radius: 9px;
}
/* Число Рейнольдса выводится по центру */
.re-box {
    display: flex;
    align-items: center;
    justify-content: center;
    background: var(--accent-light);
    text-align: center;
}
/* Проверка геометрии */
.geometry-box {
    background: #f6f8f8;
}
/* Заголовок проверки */
.geometry-box strong {

```

```

display: block;
margin-bottom: 4px;
}
/* Ошибка геометрии */
.geometry-box.invalid {
color: var(--error);
background: #fff1ef;
border-color: #f1c5bf;
}
/* Строки проверки геометрии */
.geometry-main,
.geometry-detail {
display: block;
}
/* Дополнительная информация */
.geometry-detail {
margin-top: 2px;
color: var(--muted);
white-space: nowrap;
}
/* Кнопка запуска находится у нижней границы блока */
#calculate-btn {
width: 100%;
margin-top: auto;
padding: 13px 20px;
border: 0;
border-radius: 9px;
background: var(--accent);
color: #ffffff;
font-size: 15px;
font-weight: 600;
cursor: pointer;
}
/* Цвет кнопки при наведении */
#calculate-btn:hover {
background: var(--accent-dark);
}
/* Заблокированная кнопка */
#calculate-btn.disabled {
opacity: 0.5;
cursor: default;
}
/* Блок хода расчёта */
.process {
margin-top: 20px;
padding: 20px 26px;
}
/* Основная строка блока */
.process-head {
display: flex;
align-items: center;
justify-content: space-between;

```

```

    gap: 24px;
  }
  /* Текстовая часть блока */
  .process-info {
    flex: 1;
  }
  /* Заголовок хода расчёта */
  .process-info h2 {
    margin: 0 0 5px;
    font-size: 21px;
  }
  /* Подпись под заголовком */
  .process-info p {
    margin: 0;
    color: var(--muted);
    font-size: 14px;
  }
  /* Текущее состояние расчёта */
  .status {
    display: flex;
    align-items: center;
    justify-content: center;
    flex: 0 0 180px;
    height: 46px;
    padding: 0 18px;
    border-radius: 10px;
    background: var(--accent-light);
    color: var(--accent-dark);
    font-size: 16px;
    font-weight: 700;
    line-height: 1;
    text-align: center;
  }
  /* Журнал расчёта */
  .log-details {
    margin-top: 12px;
  }
  /* Кнопка показа журнала */
  .log-details summary {
    display: inline-block;
    color: var(--accent-dark);
    font-size: 14px;
    cursor: pointer;
    user-select: none;
  }
  /* Содержимое журнала */
  .log-details pre {
    max-height: 220px;
    overflow: auto;
    margin: 10px 0 0;
    padding: 14px;
    border-radius: 8px;

```

```

background: #122522;
color: #e9f4f2;
font: 12px/1.4 Consolas, monospace;
white-space: pre-wrap;
}
/* Блок результатов скрыт до окончания расчёта */
.results {
  display: none;
  margin-top: 20px;
}
/* Заголовок результатов */
.results > .card-head {
  margin-bottom: 14px;
}
/* Результаты выводятся сеткой два на два */
.results-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}
/* Карточка одного результата */
figure {
  margin: 0;
  padding: 12px;
  border: 1px solid var(--line);
  border-radius: 10px;
  display: flex;
  flex-direction: column;
  align-items: center;
}
/* Названия всех графиков располагаются по центру */
figcaption {
  width: 100%;
  margin: 0 0 8px;
  font-size: 18px;
  font-weight: 700;
  text-align: center;
}
/* Изображение располагается по центру карточки */
figure img {
  display: block;
  width: 100%;
  max-width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: contain;
  margin: 0 auto;
}
/* На среднем экране блоки перестраиваются в одну колонку */
@media (max-width: 1000px) {
  .top-grid,
  .summary,
  .results-grid {

```

```
    grid-template-columns: 1fr;
  }
  .scheme {
    height: auto;
  }
}
/* Настройка для узких экранов */
@media (max-width: 700px) {
  .page {
    width: calc(100% - 20px);
  }
  .inputs {
    grid-template-columns: 1fr;
  }
  .process-head {
    flex-direction: column;
    align-items: stretch;
  }
  .status {
    flex-basis: auto;
    width: 100%;
  }
}
```

```
#!/bin/bash
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
CASE_DIR="$SCRIPT_DIR/case"
if ! command -v simpleFoam >/dev/null 2>&1 && [ -f /opt/openfoam9/etc/bashrc ]; then
    source /opt/openfoam9/etc/bashrc
fi
if ! command -v blockMesh >/dev/null 2>&1 || ! command -v checkMesh >/dev/null 2>&1 || !
command -v simpleFoam >/dev/null 2>&1; then
    echo "Ошибка: OpenFOAM 9 не найден" >&2
    exit 1
fi
cd "$CASE_DIR" || exit 1
bash "$SCRIPT_DIR/Clean.sh"
echo "Построение сетки"
if ! blockMesh > log.blockMesh 2>&1; then
    echo "Ошибка blockMesh" >&2
    tail -n 25 log.blockMesh >&2
    exit 1
fi
echo "Проверка сетки"
if ! checkMesh > log.checkMesh 2>&1; then
    echo "Ошибка checkMesh" >&2
    tail -n 25 log.checkMesh >&2
    exit 1
fi
if ! grep -q "Mesh OK" log.checkMesh; then
    echo "Сетка не прошла проверку" >&2
    exit 1
fi
echo "Расчёт simpleFoam"
if ! simpleFoam > log.simpleFoam 2>&1; then
    echo "Ошибка simpleFoam" >&2
    tail -n 30 log.simpleFoam >&2
    exit 1
fi
touch foam.foam
grep -E "SIMPLE solution converged|End$" log.simpleFoam | tail -n 2
```